

2025 - 2026
PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

Plateforme d'Automatisation du Support ISP
par l'Intelligence Artificielle

Réalisé par: Mohamed Mehdi Kalfat

Encadré par: M. Tarek Ben Slama

Encadrant ESPRIT: Mme Olfa Laayouni

TRITUX
Group

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous :

Nom & Prénom : **Kalfat Mohamed Mehdi**

Encadrant Entreprise

- Nom & Prénom : **Mr. Ben Slama Tarek**

Cachet & Signature



Ben Slama Tarek

[Signature]

Encadrant Académique

- Nom & Prénom : **Mme. Laayouni Olfa**

Signature

Olfa Layouni

Ce formulaire doit être rempli, signé et **scanné**

Ce formulaire doit être introduit après la page de garde

Dédicaces

*À mes chers parents,
pour leur soutien indéfectible et leurs sacrifices tout au long de mon parcours.*

*À ma famille et à mes amis,
pour leurs encouragements constants.*

*À mon encadrante académique, Mme. Olfa Laayouni,
pour sa bienveillance et ses conseils précieux.*

*À mes encadrants professionnels, M. Mohamed Amine Kilani
et M. Tarek Ben Slama,
pour leur accompagnement et leur confiance.*

*À toute l'équipe de **Tritux Group**
et à l'ensemble du corps enseignant de **ESPRIT**.*

*À une personne qui m'est chère,
qui, dans l'ombre, m'a tant aidé et soutenu,
et sans qui ce travail ne serait pas ce qu'il est aujourd'hui.*

*À tous ceux qui m'ont accompagné, de près ou de loin,
dans la réalisation de ce travail,
je dédie ce modeste travail.*

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail.

Je remercie tout particulièrement mon encadrante académique, **Mme. Olfa Laayouni**, pour la qualité de son encadrement, ses conseils avisés, sa disponibilité et la confiance qu'elle m'a accordée durant toute la durée de ce projet.

J'adresse également mes vifs remerciements à toute l'équipe de **Tritux Group** pour son accueil chaleureux, son accompagnement technique et l'environnement de travail stimulant qu'elle a su offrir, ainsi qu'à mes encadrants professionnels, **M. Mohamed Amine Kilani** et **M. Tarek Ben Slama**, pour leur suivi, leur disponibilité et leurs orientations tout au long de ce stage.

Mes remerciements s'étendent à l'ensemble du corps enseignant et administratif de **l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)** pour la formation de qualité dont j'ai bénéficié.

Enfin, je tiens à remercier les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail.

Résumé

Le support technique des fournisseurs d'accès à Internet (FAI) souffre d'un écart croissant entre la complexité des équipements d'accès (modems, ONT, passerelles domestiques) et la rapidité attendue de résolution des incidents. Les agents de premier niveau manquent souvent de l'expertise nécessaire, tandis que les ingénieurs du centre d'exploitation réseau (NOC) perdent un temps précieux à interroger manuellement les paramètres normalisés TR-069. Ce projet de fin d'études propose une **plateforme d'automatisation du support ISP** qui transforme la documentation technique passive en une intelligence actionnable. La solution combine trois piliers : un **graphe de connaissances Neo4j** interrogé par une approche **GraphRAG**, un **agent d'intelligence artificielle** propulsé par un grand modèle de langage (LLM) exécuté localement (Ollama / Qwen 2.5), et une **couche d'action MCP** (Model Context Protocol) qui exécute de véritables opérations de télégestion sur un serveur ACS conforme au standard TR-069. Le système met en œuvre une boucle fermée de remédiation — comprendre, récupérer, diagnostiquer, remédier — capable de corriger à distance les pannes courantes et d'escalader les incidents matériels. L'architecture, déployée en microservices Docker, a été validée sur un banc d'essai reproduisant douze modèles de routeurs réels.

Mots-clés : Support ISP, GraphRAG, LLM local, Neo4j, Model Context Protocol, TR-069, GenieACS, agent IA, microservices, automatisation.

Abstract

Internet Service Provider (ISP) technical support suffers from a widening gap between the complexity of access equipment and the expected speed of incident resolution. First-line agents often lack the required expertise, while Network Operations Center (NOC) engineers waste time manually querying standardized TR-069 parameters. This graduation project introduces an **ISP support automation platform** that turns passive technical documentation into actionable intelligence. The solution combines a **Neo4j knowledge graph** queried through a **GraphRAG** approach, an **AI agent** powered by a locally executed large language model (Ollama / Qwen 2.5), and an **MCP action layer** (Model Context Protocol) that performs real remote-management operations on a TR-069 compliant ACS. The system implements a closed remediation loop — understand, retrieve, diagnose, remediate — able to remotely fix common faults and escalate hardware incidents. The Docker microservices architecture was validated on a test bench reproducing twelve real router models.

Keywords: ISP support, GraphRAG, local LLM, Neo4j, Model Context Protocol, TR-069, GenieACS, AI agent, microservices, automation.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Table des figures	xi
Liste des tableaux	xii
Liste des acronymes	xiii
Introduction générale	1
1 Contexte général du projet	3
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	3
1.3 Cadre du projet	3
1.4 Enjeux du support technique télécom	4
1.5 Étude de l'existant et problématique	5
1.6 Critique de l'existant	5
1.6.1 Illustration du processus actuel	6
1.7 Solution proposée	6
1.7.1 Proposition de valeur	7
1.7.2 Étude comparative	8
1.7.3 Objectifs du projet	8
1.8 Conclusion	9
2 État de l'art et fondements techniques	10
2.1 Introduction	10

2.2	L'intelligence artificielle et ses grands domaines	10
2.3	Les grands modèles de langage (LLM)	10
2.4	La génération augmentée par récupération (RAG)	12
2.5	Du RAG vectoriel au GraphRAG	12
2.6	Le Model Context Protocol (MCP)	13
2.6.1	MCP classique, UTCP et « code mode »	14
2.7	Le standard TR-069 et le serveur ACS	15
2.7.1	Comparaison : LLM local contre LLM en nuage	15
2.7.2	Illustration : l'arborescence des paramètres TR-069	16
2.8	Neo4j et les graphes de connaissances	16
2.9	Approfondissement : architecture Transformer et attention	16
2.10	Tokenisation, plongements et similarité vectorielle	17
2.11	IA agentique et appel d'outils	17
2.12	Le format JSON-RPC	18
2.13	Conteneurisation et orchestration	18
2.14	Sécurité : hachage des identifiants	18
2.15	Approfondissements théoriques	18
2.15.1	Pourquoi combiner ces briques plutôt qu'une seule	18
2.15.2	Une brève histoire des modèles de langage	19
2.15.3	Le mécanisme d'attention, expliqué par l'exemple	19
2.15.4	Ingénierie de prompt et cadrage du modèle	20
2.15.5	Bases orientées graphe contre bases relationnelles	20
2.15.6	Le protocole CWMP en détail	20
2.15.7	La sûreté des agents autonomes	21
2.16	Illustrations et exemples des concepts	21
2.16.1	Exemple : de la phrase de l'abonné aux tokens	21
2.16.2	Exemple : hallucination sans RAG, ancrage avec GraphRAG . . .	22
2.16.3	Exemple : un parcours GraphRAG pas à pas	22
2.16.4	Exemple : un échange MCP en JSON-RPC	23
2.16.5	Exemple : un échange TR-069 (lecture puis écriture)	23

2.17 Conclusion	24
3 Analyse et spécification des besoins	25
3.1 Introduction	25
3.2 Méthodologie de conduite de projet	25
3.3 Identification des acteurs	26
3.4 Spécification des besoins fonctionnels	27
3.5 Spécification des besoins non fonctionnels	27
3.6 Diagramme de cas d'utilisation	28
3.6.1 Raffinement de cas d'utilisation	28
3.6.2 Cas d'utilisation « S'authentifier »	29
3.6.3 Cas d'utilisation « Injecter / corriger une panne »	29
3.6.4 Cas d'utilisation « Décrire un problème »	29
3.6.5 Cas d'utilisation « Superviser le parc »	29
3.6.6 Vue d'ensemble du flux fonctionnel	29
3.7 Planification et gestion des risques	31
3.7.1 Planification du projet	31
3.7.2 Analyse des risques	31
3.8 Conclusion	31
4 Conception	33
4.1 Introduction	33
4.2 Architecture globale	33
4.3 Architecture logique en couches	34
4.4 Architecture physique : microservices Docker	34
4.5 Diagramme de classes	35
4.6 Modèles de données	37
4.6.1 Modèle documentaire MongoDB	37
4.6.2 Graphe de connaissances Neo4j	37
4.7 Diagramme de séquence : résolution d'une panne	38
4.8 Scénarios de séquence complémentaires	38

4.8.1	Séquence d'authentification	38
4.8.2	Séquence d'injection de panne (NOC)	38
4.8.3	Construction et alimentation du graphe	39
4.9	Conception de la résolution rôle $\rightarrow$ chemin TR-069	39
4.10	Catalogue des rôles et modèle de pannes	40
4.11	Résolution par constructeur : preuve de la nécessité du graphe	40
4.12	Schéma documentaire MongoDB	42
4.13	Exemples détaillés de résolution	42
4.13.1	Exemple : un même correctif, trois chemins	42
4.13.2	Exemple : calcul du statut dérivé	43
4.13.3	Exemple : cheminement d'une requête dans l'architecture	44
4.14	Modélisation détaillée du graphe de connaissances	44
4.15	Conclusion	45
5	Réalisation	46
5.1	Introduction	46
5.2	Environnement de travail	46
5.2.1	Environnement matériel	46
5.2.2	Environnement logiciel et pile technologique	46
5.3	Déploiement conteneurisé	46
5.4	Implémentation des composants clés	48
5.4.1	Moteur de décision et couche GraphRAG	48
5.4.2	GenieACS : la lecture de l'état	48
5.4.3	RaDuce : le moteur de remédiation	48
5.4.4	Boucle fermée et sûreté	49
5.5	Interfaces réalisées : portail client (front office)	49
5.6	Interfaces réalisées : tableau de bord NOC (back office)	52
5.7	Scénario de démonstration de bout en bout	53
5.8	Tests et validation	53
5.9	Procédure de déploiement	53

5.10	Plan de tests et validation	56
5.11	Illustration d'une résolution tracée	57
5.12	Justification des choix technologiques	57
5.13	Scénario de bout en bout commenté	58
5.14	Exemples d'interactions et de résolutions	60
5.14.1	Exemple : un dialogue complet avec l'assistant	60
5.14.2	Exemples par famille de pannes	60
5.14.3	Exemple : l'observabilité offerte au NOC	62
5.15	Parcours détaillé des interfaces	62
5.15.1	Portail abonné, écran par écran	62
5.15.2	Tableau de bord NOC, écran par écran	63
5.15.3	Environnement technique de la réalisation	63
5.16	Conclusion	63
6	Évaluation, discussion et perspectives	65
6.1	Introduction	65
6.2	Bilan des réalisations	65
6.3	Confrontation aux objectifs	65
6.4	Discussion des limites	65
6.5	Analyse de sécurité	66
6.6	Scalabilité et maintenabilité	66
6.7	Perspectives	67
6.8	Conclusion	67
6.9	Retour d'expérience	68
6.10	Synthèse des contributions	68
6.11	Positionnement par rapport aux approches existantes	68
6.12	Enseignements généralisables	69
	Conclusion générale et perspectives	70
	Bibliographie	72

A	Orchestration des microservices (Docker Compose)	73
B	Extrait du moteur de résolution et de diagnostic	77
C	Extrait du moteur de remédiation (RaDuce)	80
D	Extrait de la résolution GraphRAG (neo4j-agent)	82
E	Extrait d'initialisation des données (seed)	84
F	Glossaire	86

Table des figures

1.1	Le modem-routeur : point de jonction entre le réseau de l'opérateur et le réseau local de l'abonné.	4
1.2	La boucle fermée d'automatisation : comprendre, récupérer, diagnostiquer, remédier.	7
2.1	Architecture et principe de fonctionnement d'un grand modèle de langage.	11
2.2	Architecture d'une chaîne RAG : récupérer, augmenter, générer.	12
2.3	Le GraphRAG : de la question à la réponse ancrée dans le graphe.	13
2.4	Le Model Context Protocol : hôte, client, serveur et exposition d'outils.	14
3.1	Diagramme de cas d'utilisation de la plateforme.	28
4.1	Architecture globale de la plateforme.	34
4.2	Architecture logique en couches.	34
4.3	Architecture physique — conteneurs Docker.	35
4.4	Vue microservices du déploiement.	35
4.5	Diagramme de classes de la plateforme.	36
4.6	Schéma du graphe de connaissances Neo4j : (:Router)-[:HAS_PARAM {role}]->(:Parameter).	37
4.7	Diagramme de séquence de la résolution d'une panne.	38
5.1	Outils et technologies mobilisés.	47
5.2	Supervision des conteneurs sous Docker Desktop.	47
5.3	Portail client : authentification et page d'accueil.	49
5.4	Page promotionnelle du portail client.	50
5.5	Processus de vérification de l'abonné par e-mail.	50
5.6	Assistant conversationnel : diagnostic et correction en langage naturel.	51
5.7	Suivi des tickets, notifications et statistiques côté abonné.	51
5.8	Notification par courriel d'un problème détecté sur le routeur.	52

5.9	Gestion des abonnés côté NOC.	52
5.10	Supervision et gestion des incidents côté NOC.	52
5.11	Observabilité et tests : raisonnement de l'IA et injection de pannes. . . .	54
5.12	Statistiques détaillées par routeur.	55
5.13	Scénario de bout en bout : de la plainte à la résolution autonome.	55

Liste des tableaux

1.1	Étude comparative des approches de support.	8
2.1	RAG vectoriel classique vs GraphRAG.	13
2.2	LLM local (retenu) contre LLM en nuage.	15
3.1	Itérations du développement selon le 2TUP.	26
3.2	Besoins fonctionnels par acteur.	27
3.3	Besoins non fonctionnels.	27
3.4	Description du cas d'utilisation « Diagnostiquer & corriger une panne ». .	28
3.5	Description du cas d'utilisation « S'authentifier ».	29
3.6	Description du cas d'utilisation « Injecter / corriger une panne » (NOC). .	29
3.7	Description du cas d'utilisation « Décrire un problème ».	30
3.8	Description du cas d'utilisation « Superviser le parc » (NOC).	30
3.9	Phases du projet et livrables.	31
3.10	Principaux risques et mesures d'atténuation.	32
4.1	Microservices de la plateforme.	36
4.2	Catalogue des rôles canoniques.	40
4.3	Modèle de pannes et correctifs associés.	41
4.4	Résolution du rôle <code>vlan_id</code> en chemin TR-069 pour les douze modèles. .	41
4.5	Un même rôle logique résolu en trois chemins TR-069 distincts.	43
4.6	Statut dérivé avant et après application du correctif.	43
5.1	Plan de tests fonctionnels de la boucle de remédiation.	56
5.2	Exemple de dialogue abonné ↔ assistant (coupure VLAN).	61
5.3	Environnement technique du projet.	64
6.1	Objectifs initiaux et niveau de réalisation.	66

Liste des acronymes

Acronyme	Signification
ACS	Auto-Configuration Server (serveur d'auto-configuration)
API	Application Programming Interface
CWMP	CPE WAN Management Protocol (TR-069)
CPE	Customer Premises Equipment (équipement chez l'abonné)
FAI	Fournisseur d'Accès à Internet
GPON	Gigabit-capable Passive Optical Network
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model (grand modèle de langage)
MCP	Model Context Protocol
MTTR	Mean Time To Repair (temps moyen de résolution)
NLP	Natural Language Processing (traitement du langage naturel)
NOC	Network Operations Center (centre d'exploitation réseau)
ONT	Optical Network Terminal
PPP	Point-to-Point Protocol
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
TR-069	Technical Report 069 (Broadband Forum)
UML	Unified Modeling Language
UTCP	Universal Tool Calling Protocol
VLAN	Virtual Local Area Network
2TUP	Two Track Unified Process

Introduction générale

La transformation numérique du secteur des télécommunications s'accompagne d'une complexification continue des réseaux d'accès. Les fournisseurs d'accès à Internet (FAI) déploient une grande diversité d'équipements chez leurs abonnés — modems xDSL, terminaux optiques (ONT/GPON), passerelles domestiques multiservices — provenant de constructeurs différents et exposant chacun des centaines de paramètres de configuration. Dans ce contexte, la qualité et la rapidité du support technique deviennent un facteur clé de compétitivité et de fidélisation.

Pourtant, le support de premier niveau demeure largement manuel et cloisonné. Les agents Tier 1 ne disposent pas toujours de l'expertise nécessaire pour traiter les pannes complexes, la documentation technique est dispersée dans d'innombrables manuels et fichiers PDF, et les ingénieurs du NOC consacrent un temps considérable à interroger un par un les paramètres normalisés des équipements. Il en résulte des délais de résolution élevés, des coûts d'exploitation importants et une expérience abonné dégradée.

Parallèlement, les progrès récents de l'intelligence artificielle — en particulier les grands modèles de langage (LLM), la génération augmentée par récupération (RAG) et les protocoles standardisés d'orchestration d'outils — ouvrent la voie à une nouvelle génération de systèmes de support capables non seulement de *comprendre* une plainte formulée en langage naturel, mais aussi d'*agir* concrètement sur les équipements.

C'est dans cette perspective que s'inscrit le présent projet de fin d'études, réalisé au sein de **Tritux Group**. Il vise à concevoir et réaliser une plateforme intelligente et auto-réparatrice pour le support ISP, qui associe un graphe de connaissances (GraphRAG), un agent IA fondé sur un LLM local, et une couche d'action fondée sur le Model Context Protocol (MCP) pilotant un serveur ACS conforme au standard TR-069.

L'originalité de la démarche tient à ce qu'elle ne se contente pas d'ajouter une couche conversationnelle au-dessus des outils existants, mais qu'elle referme la boucle entre la *compréhension* du problème et l'*action* sur l'équipement. Là où la plupart des assistants s'arrêtent à la formulation d'une réponse, la plateforme proposée exécute une correction réelle et en vérifie l'effet. Cette ambition impose des exigences fortes : la connaissance mobilisée doit être exacte et non approximative, les actions doivent être sûres et réversibles, et l'ensemble doit demeurer confidentiel et maîtrisé, d'où le choix d'une intelligence artificielle exécutée localement.

Le fil conducteur de ce travail est ainsi la transformation d'une connaissance *passive* — dispersée dans des documents — en une connaissance *active*, structurée dans un graphe

et directement exploitable par un agent autonome. Ce déplacement, de la donnée qui se lit vers la donnée qui agit, constitue le cœur de la contribution. Le présent rapport rend compte de ce cheminement, de l'analyse du besoin jusqu'à la validation de la solution. Il s'attache à expliquer non seulement *ce qui* a été réalisé, mais aussi *pourquoi* chaque choix a été fait, afin que le lecteur puisse apprécier la cohérence d'ensemble de la démarche autant que ses résultats.

Ce rapport est organisé en cinq chapitres. Le **premier chapitre** présente le contexte général du projet, l'organisme d'accueil, la problématique et la solution proposée. Le **deuxième chapitre** constitue un état de l'art des fondements techniques mobilisés (LLM, RAG, GraphRAG, MCP, TR-069). Le **troisième chapitre** est consacré à l'analyse et à la spécification des besoins ainsi qu'à la méthodologie de conduite de projet. Le **quatrième chapitre** détaille la conception de la solution (architectures, diagrammes UML, modèles de données). Enfin, le **cinquième chapitre** décrit la réalisation, les interfaces développées et la validation du système. Une conclusion générale dresse le bilan du travail accompli et esquisse les perspectives d'évolution.

CHAPITRE 1

Contexte général du projet

1.1 Introduction

Ce premier chapitre pose le cadre général du projet. Il présente l'organisme d'accueil, situe le projet dans son environnement métier, analyse l'existant et sa problématique, puis expose la solution proposée et la méthodologie de conduite de projet adoptée.

1.2 Présentation de l'organisme d'accueil

Ce projet de fin d'études a été réalisé au sein de **Tritux Group**, entreprise internationale spécialisée dans le développement logiciel, la transformation digitale, le Cloud, le DevOps et l'intelligence artificielle. Fondée en 2006, Tritux dispose de bureaux en Tunisie et en France et accompagne des clients dans plusieurs pays.

L'entreprise conçoit des solutions innovantes pour les secteurs des télécommunications, de la finance, du secteur public et des entreprises. Grâce à son expertise technique et à son approche orientée innovation, Tritux réalise des projets à forte valeur ajoutée répondant aux besoins de la transformation numérique. C'est dans le cadre de son activité télécom, et plus particulièrement de la modernisation des outils de support, que s'inscrit le présent projet. Ce positionnement à l'intersection de l'intelligence artificielle et des télécommunications fait de Tritux un cadre particulièrement adapté à un projet qui mobilise précisément ces deux expertises : la maîtrise des protocoles de gestion réseau d'une part, et celle de technologies modernes d'IA d'autre part.

1.3 Cadre du projet

Un fournisseur d'accès à Internet gère un vaste parc d'équipements installés chez les abonnés, désignés dans la terminologie normalisée par le terme *CPE* (Customer Premises Equipment) : modems xDSL, terminaux optiques (ONT) sur les réseaux GPON, passerelles domestiques. La figure 1.1 illustre la place du routeur/modem dans le réseau local de l'abonné, à l'interface entre le réseau d'accès de l'opérateur et les équipements domestiques.

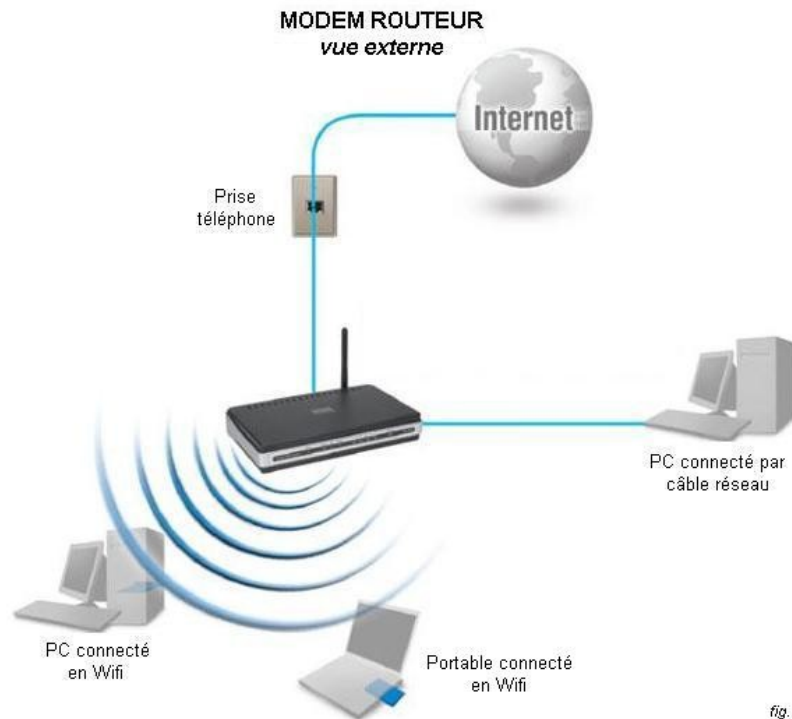


fig. 2

Figure 1.1. Le modem-routeur : point de jonction entre le réseau de l'opérateur et le réseau local de l'abonné.

La gestion à distance de ces équipements est standardisée par le protocole **TR-069** (également appelé CWMP, *CPE WAN Management Protocol*), défini par le Broadband Forum. Ce protocole permet à un serveur d'auto-configuration (*ACS*) de lire et d'écrire, à distance, les paramètres d'un équipement selon une arborescence normalisée (par exemple `InternetGatewayDevice.WANDevice.{i}...`). Le support technique d'un FAI s'appuie donc, en théorie, sur cette capacité de télégestion ; en pratique, son exploitation reste largement manuelle.

1.4 Enjeux du support technique télécom

Le support technique n'est pas une fonction périphérique pour un FAI : il est au cœur de la relation client et pèse directement sur la rentabilité. Chaque appel de niveau 1 a un coût ; chaque déplacement de technicien en a un bien supérieur ; et chaque incident mal résolu dégrade la satisfaction et augmente le risque de résiliation. Trois enjeux se conjuguent. L'**enjeu économique** : réduire le volume d'interventions humaines et les délais. L'**enjeu de qualité** : offrir une résolution rapide, homogène et disponible en permanence. L'**enjeu de connaissance** : capitaliser l'expertise technique, souvent détenue par quelques experts, dans un actif réutilisable et non tributaire des personnes. C'est à ces trois enjeux que la plateforme entend répondre simultanément.

1.5 Étude de l'existant et problématique

L'analyse du processus de support actuel fait apparaître un **écart croissant entre la complexité des équipements et la rapidité de résolution attendue**. Trois difficultés structurelles ont été identifiées :

- **Agents de niveau 1 (Tier 1)** : ils manquent des connaissances techniques approfondies nécessaires pour traiter les pannes complexes, ce qui conduit à des transferts fréquents vers les niveaux supérieurs.
- **Ingénieurs Tier 2 / NOC** : ils perdent un temps considérable à interroger manuellement les paramètres TR-069, équipement par équipement, constructeur par constructeur.
- **Documentation dispersée** : l'information technique est éclatée dans d'innombrables manuels, wikis internes et fichiers PDF, imposant une recherche fastidieuse et source d'erreurs.

Problématique. Comment transformer la documentation technique passive et dispersée d'un opérateur en une intelligence *actionnable*, capable de diagnostiquer et de corriger automatiquement les pannes des équipements d'abonnés, tout en préservant la confidentialité des données et en garantissant la fiabilité des actions exécutées ?

1.6 Critique de l'existant

Deux familles de solutions existantes ont été examinées. Le **support traditionnel** (humain, assisté d'outils ACS classiques) est fiable mais lent, coûteux et fortement dépendant de l'expertise individuelle. Les **chatbots fondés sur un RAG vectoriel classique** savent répondre à des questions documentaires mais restent cantonnés à un rôle conversationnel : ils n'agissent pas sur le matériel, perdent les dépendances techniques entre paramètres et sont sujets aux hallucinations. Aucune de ces approches ne referme la boucle entre le *diagnostic* et l'*action corrective* réelle. On peut résumer cette critique par une image : le support traditionnel dispose des *maines* (l'accès à l'ACS) mais pas toujours du *cerveau* disponible et disponible en continu ; le chatbot RAG classique dispose d'une forme de *parole* mais n'a pas de *maines* pour agir. La solution recherchée doit réunir les trois : la parole pour dialoguer, le cerveau pour raisonner sur une connaissance exacte, et les mains pour corriger. C'est cette réunion, et non l'une de ces facultés prise isolément, qui constitue la véritable difficulté — et l'apport — du projet.

1.6.1 Illustration du processus actuel

Pour mesurer l'écart, comparons le traitement d'une simple coupure liée à un VLAN mal configuré. Dans le **processus actuel**, l'abonné appelle le support ; l'agent Tier 1, faute d'accès ou de compétence, ouvre un ticket et le transfère ; un ingénieur Tier 2 se connecte à l'ACS, cherche le bon modèle dans la documentation, identifie le chemin propre au constructeur, lit la valeur, la corrige, puis rappelle l'abonné. Ce cycle peut s'étaler sur plusieurs heures. Dans le **processus cible**, l'abonné décrit sa panne en langage naturel et l'agent réalise l'ensemble — lecture, diagnostic, correction, vérification — en quelques secondes, sans intervention humaine, l'ingénieur n'étant sollicité que pour les pannes réellement physiques.

Exemple

Ce que l'exemple révèle. La lenteur du processus actuel ne vient pas d'un manque d'outils (l'ACS existe), mais du fait que la *connaissance* nécessaire pour s'en servir est dispersée et non exécutable. Le projet ne remplace pas l'ACS : il rend sa connaissance directement actionnable par une IA.

1.7 Solution proposée

Pour répondre à cette problématique, nous proposons une plateforme qui fait passer le support « d'une boucle avec humain dans la boucle » à une **autonomie pilotée par l'IA**, articulée autour de quatre étapes formant une boucle fermée (figure 1.2) :

1. **Comprendre** : analyser la plainte de l'abonné exprimée en langage naturel ;
2. **Récupérer** : interroger un graphe de connaissances pour retrouver le contexte technique exact de l'équipement concerné ;
3. **Diagnostiquer** : analyser en autonomie les paramètres temps réel de l'équipement afin d'identifier la panne ;
4. **Remédier** : exécuter des actions correctives réelles sur l'équipement via un serveur ACS, puis vérifier la résolution.

Chacune de ces étapes mérite d'être précisée. **Comprendre** ne se limite pas à reconnaître des mots-clés : l'agent doit saisir l'*intention* derrière une formulation souvent imprécise, identifier l'abonné et le rattacher à son équipement. **Récupérer** signifie aller chercher, dans le graphe, non pas un texte approximativement pertinent mais le fait technique exact — le chemin d'un paramètre, sa valeur attendue, son caractère modifiable.

Diagnostiquer consiste à confronter l'état réel de l'équipement aux règles métier pour nommer précisément la panne, en distinguant ce qui est réparable à distance de ce qui relève d'une intervention physique. **Remédier**, enfin, n'est pas une simple suggestion : c'est l'écriture effective d'une valeur au bon endroit, suivie d'une vérification qui seule permet de déclarer l'incident résolu. C'est l'articulation rigoureuse de ces quatre temps qui distingue une véritable automatisation d'un simple assistant conversationnel.

Cette boucle possède une propriété remarquable : elle est *honnête*. À aucun moment le système ne prétend avoir résolu un incident qu'il n'a pas effectivement corrigé, puisque la dernière étape relit l'état réel. Cette honnêteté, souvent négligée dans les démonstrateurs, est en réalité la condition de l'acceptabilité d'un tel système par un opérateur : mieux vaut une escalade franche qu'une fausse résolution qui reviendrait, aggravée, quelques heures plus tard.

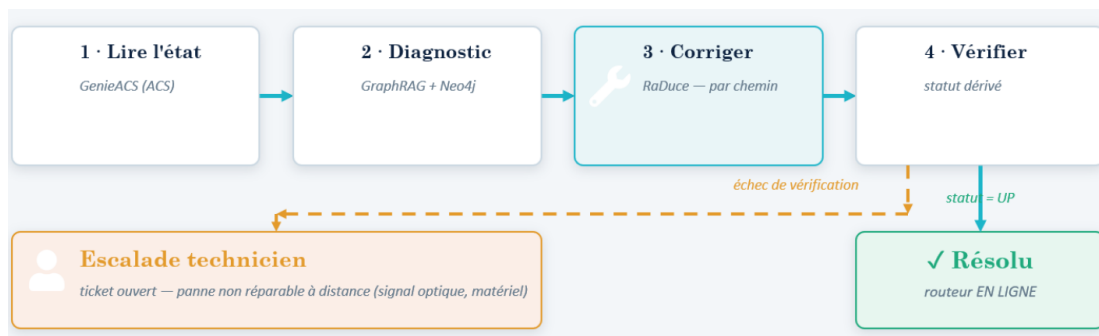


Figure 1.2. La boucle fermée d'automatisation : comprendre, récupérer, diagnostiquer, remédier.

1.7.1 Proposition de valeur

La valeur du projet repose sur une observation simple : les opérateurs possèdent déjà une **mine d'or** de documentation (wikis, procédures, tableurs de paramètres), mais cette donnée est « morte », figée dans des PDF ou cloisonnée dans des colonnes de base de données. La contribution de la plateforme est de **transformer cette documentation passive en intelligence actionnable et exécutable**. Autrement dit, la valeur ne provient pas de la création d'une nouvelle donnée, mais de l'activation d'une donnée qui dort déjà. C'est une proposition à fort retour sur investissement : l'opérateur ne repart pas de zéro, il rentabilise un patrimoine informationnel existant. Ce constat oriente toute la conception : plutôt que d'entraîner un modèle sur mesure, on structure et on exécute la connaissance disponible. Ce parti pris a une conséquence stratégique. En s'appuyant sur la connaissance existante et sur des standards ouverts (TR-069, MCP), la plateforme reste *indépendante* d'un fournisseur particulier de modèle ou d'équipement. L'opérateur n'est captif ni d'un constructeur — puisque le graphe absorbe la diversité matérielle —

ni d'un fournisseur d'IA — puisque le modèle, exécuté localement, peut être remplacé. Cette indépendance est un atout de long terme, dans un secteur où les technologies évoluent vite et où la dépendance à un tiers peut devenir un risque.

1.7.2 Étude comparative

Le tableau 1.1 positionne notre solution par rapport aux approches existantes selon les critères jugés déterminants pour le support télécom.

Table 1.1. Étude comparative des approches de support.

Critère	Support traditionnel	Chatbot RAG classique	Notre solution
Autonomie de résolution	✗	~	✓
Contexte relié (GraphRAG)	✗	✗	✓
Action réelle sur le matériel (TR-069)	✗	✗	✓
IA locale & confidentialité	~	✗	✓
Observabilité / supervision NOC	~	✗	✓
Robustesse aux mises à jour firmware	✗	~	✓
Temps de résolution (MTTR)	Élevé	Moyen	Faible

1.7.3 Objectifs du projet

Les objectifs se déclinent en deux volets. Sur le plan **métier**, il s'agit de réduire drastiquement les délais de résolution des tickets (MTTR), de diminuer les coûts d'exploitation de

l'opérateur et d'améliorer l'expérience et la fidélisation des abonnés. Sur le plan **technique**, les objectifs sont de construire un moteur d'IA agentique propulsé par des LLM locaux (Ollama / Qwen 2.5), de modéliser la documentation des équipements sous forme de graphe de connaissances Neo4j, et de bâtir une couche d'abstraction MCP exécutant des outils en toute sécurité sur un serveur GenieACS.

1.8 Conclusion

Ce chapitre a défini le contexte métier du projet, mis en évidence les limites du support ISP actuel et présenté la solution proposée : une plateforme d'automatisation en boucle fermée. Le chapitre suivant présente les fondements techniques sur lesquels repose cette solution.

CHAPITRE 2

État de l’art et fondements techniques

2.1 Introduction

La solution proposée mobilise plusieurs briques technologiques récentes. Ce chapitre en présente les fondements : les grands modèles de langage et leur exécution locale, la génération augmentée par récupération (RAG) et son évolution vers le GraphRAG, le protocole d’orchestration d’outils MCP et sa variante UTCP « code mode », ainsi que le standard de télégestion TR-069 et les graphes de connaissances Neo4j.

2.2 L’intelligence artificielle et ses grands domaines

L’intelligence artificielle regroupe plusieurs domaines complémentaires. Le **traitement du langage naturel** (NLP) vise à faire comprendre et générer le langage humain par la machine. La **vision par ordinateur** (CV) s’attache à interpréter images et vidéos. Les **grands modèles de langage** (LLM), sous-ensemble du NLP fondé sur l’apprentissage profond, constituent le cœur de notre agent. Dans ce projet, seuls le NLP et les LLM sont mobilisés ; la vision par ordinateur est citée uniquement pour situer le domaine. Il importe de comprendre que ces domaines ne s’opposent pas mais se complètent : un système d’IA moderne combine souvent plusieurs briques. Dans notre cas, la brique linguistique est centrale car l’interaction avec l’abonné et la compréhension de la documentation technique sont, par nature, des problèmes de langage. Le choix de concentrer l’effort sur le NLP et les LLM découle donc directement de la nature du problème traité : transformer des mots — une plainte, une procédure, un manuel — en décisions et en actions.

2.3 Les grands modèles de langage (LLM)

Un LLM est un réseau de neurones de type *Transformer* entraîné sur d’immenses corpus textuels. Son principe de fonctionnement repose sur quatre étapes : la **tokenisation** (découpage du texte en unités), la **prédiction** du token suivant le plus probable de manière itérative, le mécanisme d’**attention** qui pondère le contexte pertinent grâce à

des milliards de paramètres, et enfin la **génération** d'une réponse cohérente. La figure 2.1 schématise cette architecture.

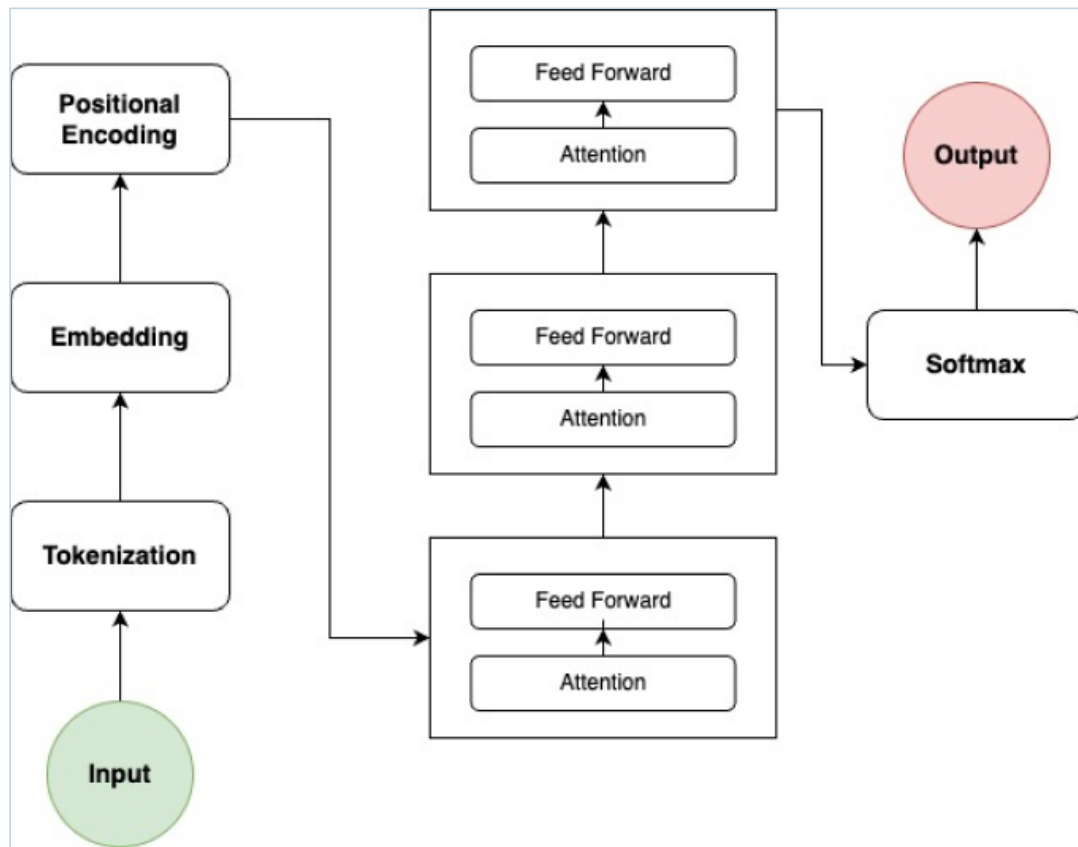


Figure 2.1. Architecture et principe de fonctionnement d'un grand modèle de langage.

Pourquoi un LLM local ? Dans ce projet, le modèle Qwen 2.5 est exécuté localement via Ollama. Ce choix garantit la **confidentialité** (aucune donnée abonné n'est envoyée vers un cloud tiers), maîtrise la **latence et les coûts** (pas d'appel d'API facturé) et assure la **souveraineté** d'un modèle léger exploité de bout en bout. La contrepartie — le risque d'hallucination d'un LLM isolé — est précisément ce que corrige le RAG.

Il faut souligner que « exécuter un modèle localement » ne signifie pas se priver de puissance : les modèles récents de taille modérée, tels que Qwen 2.5, atteignent un niveau de compréhension suffisant pour des tâches ciblées, surtout lorsqu'on leur fournit un contexte précis. La stratégie retenue consiste donc à ne pas demander au modèle de tout connaître, mais de bien raisonner sur ce qu'on lui donne — ce qui déplace la difficulté depuis la taille du modèle vers la qualité de la récupération. C'est exactement le rôle du GraphRAG.

2.4 La génération augmentée par récupération (RAG)

Le **RAG** (Retrieval-Augmented Generation) consiste, au lieu de laisser le LLM répondre « de mémoire », à récupérer d'abord des informations pertinentes dans une base de connaissances externe, puis à générer la réponse à partir de ces faits. Le pipeline comporte trois temps (figure 2.2) : *récupérer* les passages utiles, les *augmenter* dans le contexte du modèle, puis *générer* une réponse ancrée dans des données réelles. Le RAG réduit ainsi les hallucinations, exploite des données à jour et privées, et permet de tracer les sources, le tout sans réentraîner le modèle. Cette dernière propriété est capitale en contexte industriel : réentraîner un grand modèle est coûteux, lent et difficile à maintenir à jour au rythme des évolutions matérielles. Le RAG offre une alternative élégante en séparant clairement deux préoccupations : d'un côté la *capacité de raisonnement*, portée par le modèle et stable dans le temps ; de l'autre la *connaissance métier*, portée par la base externe et mise à jour indépendamment. Cette séparation est un principe d'ingénierie que notre architecture pousse jusqu'à son terme en confiant toute la connaissance au graphe.

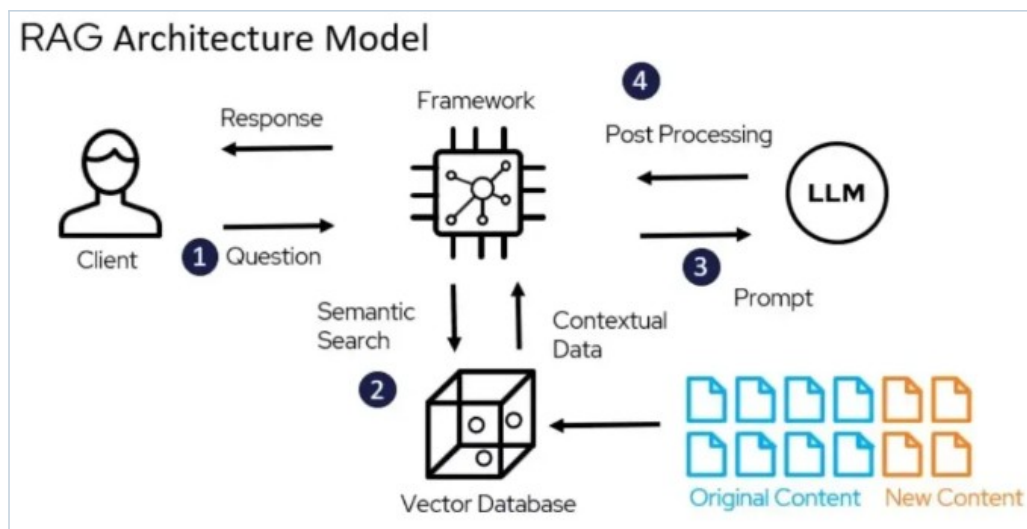


Figure 2.2. Architecture d'une chaîne RAG : récupérer, augmenter, générer.

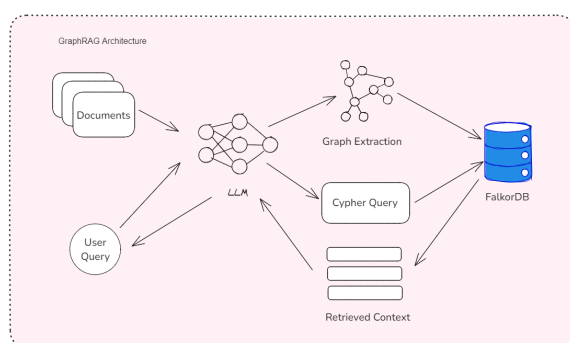
2.5 Du RAG vectoriel au GraphRAG

Le RAG classique s'appuie sur une recherche par **similarité vectorielle** (*embeddings*) : le texte est découpé en fragments arbitraires, ce qui brise les dépendances techniques et fait perdre la vue d'ensemble systémique. Le **GraphRAG** remplace cette recherche par un **parcours de graphe** : la connaissance est modélisée sous forme d'entités et de

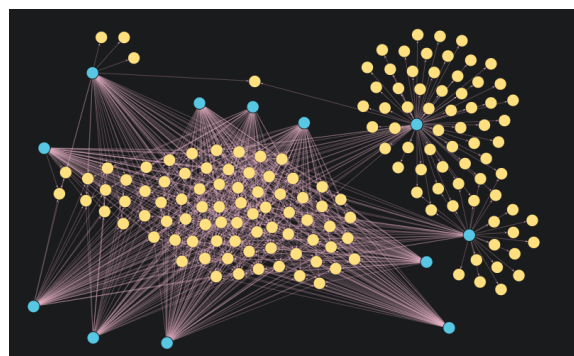
relations, et la récupération suit ces relations pour préserver les dépendances (figures 2.3a et 2.3b). Le tableau 2.1 synthétise les différences.

Table 2.1. RAG vectoriel classique vs GraphRAG.

Critère	RAG vectoriel	GraphRAG (retenu)
Recherche	Similarité de mots-clés / vecteurs	Parcours des relations du graphe
Découpage	Fragments de texte arbitraires	Entités et liens structurés
Dépendances	Souvent rompues entre fragments	Préservées (WAN → WLAN → DHCP)
Raisonnement	Vue locale, ponctuelle	Vue systémique, pas à pas



(a) Principe du GraphRAG.



(b) Chaîne GraphRAG complète.

Figure 2.3. Le GraphRAG : de la question à la réponse ancrée dans le graphe.

Pertinence pour le télécom. Une panne en amont (interface WAN) impacte les configurations en aval (WLAN, DHCP). Le graphe conserve ces liens systémiques de manière explicite, là où un simple index vectoriel les perdrait. C'est la raison du choix du GraphRAG dans ce projet.

2.6 Le Model Context Protocol (MCP)

Le **MCP** (Model Context Protocol) est un standard qui connecte un LLM à des outils et des données externes de façon uniforme, à la manière d'un « port USB » pour les capacités d'un agent (figure 2.4). Son architecture met en relation un *hôte* (le LLM qui raisonne), un *client* MCP (qui ouvre la session), un *serveur* MCP (qui expose les outils) et l'*équipement* cible. Les échanges reposent sur des messages **JSON-RPC 2.0** transportés soit par **stdio** (entrée/sortie standard, pour un serveur local), soit par HTTP en réseau. Deux méthodes clés structurent le dialogue : **tools/list** (l'agent **découvre** les outils

disponibles) et `tools/call` (l’agent **exécute** un outil). L’intérêt d’un tel standard est l’*interopérabilité* : un même agent peut, sans code spécifique, dialoguer avec des serveurs d’outils hétérogènes dès lors qu’ils respectent le protocole. Cette propriété est précieuse dans notre contexte, où coexistent plusieurs services d’action (lecture ACS, écriture ACS, gestion des abonnés). Plutôt que d’écrire une intégration sur mesure pour chacun, on les expose de manière uniforme, et l’agent les découvre dynamiquement. On retrouve ici l’esprit qui a fait le succès des interfaces standardisées en informatique : réduire le couplage en s’accordant sur un contrat commun.

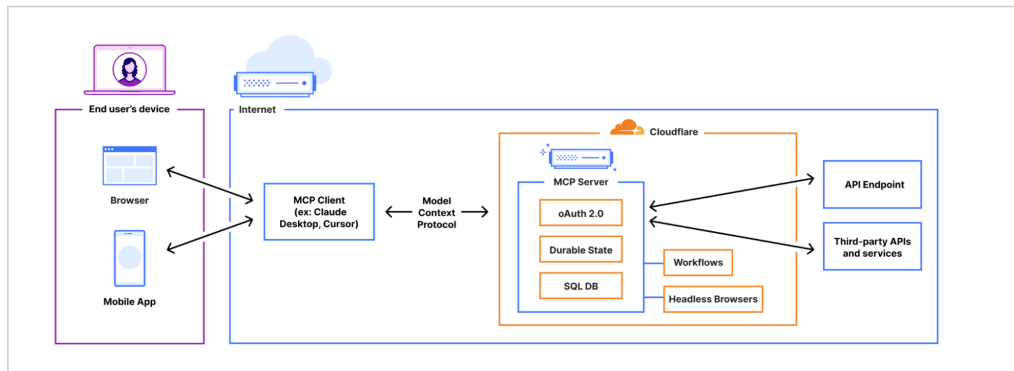


Figure 2.4. Le Model Context Protocol : hôte, client, serveur et exposition d’outils.

2.6.1 MCP classique, UTCP et « code mode »

Le MCP classique établit une session persistante dans laquelle chaque appel d’outil constitue un aller-retour via le modèle, le serveur jouant le rôle d’intermédiaire. L’**UTCP** (Universal Tool Calling Protocol) propose une alternative : un « manuel » JSON décrit chaque outil avec son URL, et l’agent appelle directement le point d’accès HTTP, sans proxy. Le « **code mode** » pousse la logique plus loin : l’agent écrit un court programme qui orchestre plusieurs appels d’outils (boucles, conditions, enchaînement), exécuté dans un bac à sable. Le choix entre ces styles n’est pas idéologique mais pragmatique. MCP apporte une découverte dynamique et une uniformité précieuses lorsque les outils sont nombreux et variés ; l’approche « code mode » réduit la cérémonie lorsqu’un simple appel HTTP suffit. Les faire coexister, plutôt que d’imposer l’un partout, illustre une posture d’ingénierie mesurée : choisir l’outil adapté à chaque besoin plutôt que d’appliquer une solution unique par principe.

Pourquoi UTCP « code mode » ? Enchaîner plusieurs outils en un seul script réduit fortement la latence et la consommation de tokens — un atout décisif pour un petit LLM local. Nos services exposent déjà un manuel UTCP (point d’accès `/tools`), et la séquence *diagnostic* → *correctif* → *vérification* s’écrit alors comme une routine unique, robuste et sûre (seules des fonctions pré-validées sont appelées).

2.7 Le standard TR-069 et le serveur ACS

Le **TR-069**/CWMP normalise la télégestion des CPE. Un serveur d’auto-configuration (ACS) tel que **GenieACS** peut lire (*GetParameterValues*) et écrire (*SetParameterValues*) les paramètres d’un équipement selon une arborescence hiérarchique. Un point crucial pour ce projet est que le **chemin exact** d’un même paramètre logique diffère selon le constructeur : le VLAN WAN s’écrit par exemple `...X_HW_VLAN` chez Huawei, `...X_TP_VID` chez TP-Link et `...VLANIDMark` chez Nokia. La connaissance de ces chemins, portée par le graphe, est donc indispensable à toute action corrective.

2.7.1 Comparaison : LLM local contre LLM en nuage

Le choix d’un LLM local n’est pas neutre ; il résulte d’un arbitrage explicite (tableau 2.2).

Table 2.2. LLM local (retenu) contre LLM en nuage.

Critère	LLM local (Ollama/Qwen)	LLM en nuage
Confidentialité	Données jamais externalisées	Données envoyées à un tiers
Coût	Pas de facturation par appel	Coût par token
Latence	Maîtrisée, sans réseau externe	Dépend du fournisseur
Puissance brute	Modèle plus léger	Modèles très volumineux
Souveraineté	Totale	Dépendance externe

La moindre puissance brute d’un modèle local est compensée par le GraphRAG, qui n’exige pas du modèle qu’il « sache » tout, mais seulement qu’il raisonne sur un contexte exact et compact.

2.7.2 Illustration : l’arborescence des paramètres TR-069

Les paramètres TR-069 sont organisés en arbre. Un même paramètre logique se situe à des emplacements voisins mais distincts selon le constructeur, comme l’illustre l’exemple suivant pour le VLAN WAN :

```
1 InternetGatewayDevice.WANDevice.1.WANConnectionDevice.1.WANPPPoEConnection.1.  
2   |-- Huawei   : X_HW_VLAN  
3   |-- TP-Link  : X_TP_VID  
4   |-- Nokia   : VLANIDMark
```

Listing 2.1. Emplacement du VLAN WAN selon le constructeur (illustratif).

Cette variabilité, invisible dans une documentation en prose, devient explicite et exploitable une fois modélisée dans le graphe.

2.8 Neo4j et les graphes de connaissances

Neo4j est une base de données orientée graphe qui stocke l’information sous forme de nœuds et de relations typées, interrogeables via le langage **Cypher**. Dans notre solution, chaque modèle de routeur et chacun de ses paramètres deviennent des nœuds reliés par des relations porteuses de propriétés (rôle, bande, éditabilité). Cette modélisation, détaillée au chapitre 4, est le support du GraphRAG.

2.9 Approfondissement : architecture Transformer et attention

Le succès des LLM repose sur l’architecture **Transformer**. Contrairement aux réseaux récurrents, celle-ci traite l’ensemble d’une séquence en parallèle grâce au mécanisme d’**auto-attention** (*self-attention*). Pour chaque token, l’attention calcule un score de pertinence vis-à-vis de tous les autres tokens du contexte, permettant au modèle de pondérer dynamiquement les dépendances, même à longue portée. Empilées sur de nombreuses couches et dotées de milliards de paramètres, ces têtes d’attention confèrent au modèle sa capacité de généralisation. La **fenêtre de contexte** — le nombre maximal de tokens pris en compte simultanément — constitue une contrainte pratique importante, particulièrement pour un modèle local léger comme Qwen 2.5 ; c’est l’une des raisons pour lesquelles la récupération ciblée du GraphRAG, qui n’injecte que les faits pertinents, est préférable à l’inondation du contexte.

2.10 Tokenisation, plongements et similarité vectorielle

Avant tout traitement, le texte est converti en **tokens** puis en **plongements** (*embeddings*) : des vecteurs de grande dimension capturant le sens sémantique. Dans un RAG vectoriel classique, la récupération repose sur la **similarité cosinus** entre le vecteur de la question et ceux des fragments indexés. Cette approche, purement statistique, ignore la structure : deux paramètres techniquement liés (par exemple une interface WAN et le service DHCP qui en dépend) peuvent se retrouver éloignés dans l’espace vectoriel. Le GraphRAG substitue à cette similarité une **navigation explicite dans les relations**, ce qui préserve les dépendances d’ingénierie. Concrètement, là où un RAG vectoriel répondrait « voici trois fragments de texte qui ressemblent à votre question », un GraphRAG répond « voici le paramètre exact, son chemin, son caractère éditable, et les éléments dont il dépend ». La première réponse demande encore un travail d’interprétation ; la seconde est directement exploitable par un agent qui doit agir. Cette différence de nature — du *ressemblant* vers l’*exact* — est ce qui rend le GraphRAG adapté à une automatisation sûre. On peut pousser l’analyse plus loin. Un index vectoriel traite chaque fragment de connaissance comme une île isolée, reliée aux autres uniquement par la ressemblance de surface. Or la connaissance technique est faite de *dépendances* : un paramètre n’a de sens qu’en relation avec d’autres (l’état PPP conditionne l’utilité de vérifier le VLAN, la puissance optique prime sur toute action logicielle). Le graphe encode nativement ces dépendances sous forme de relations typées ; les parcourir, c’est raisonner comme le ferait un expert qui suit un fil de causes et de conséquences. C’est pourquoi, dans ce projet, le graphe n’est pas un simple magasin de faits mais le support même du raisonnement.

2.11 IA agentique et appel d’outils

Un **agent** d’intelligence artificielle ne se contente pas de générer du texte : il *perçoit* un état, *décide* d’une action et l’*exécute* via des outils, puis observe le résultat et itère. Cette boucle « perception–décision–action » est rendue possible par l’**appel d’outils** (*tool calling*), par lequel le modèle sélectionne une fonction externe et fournit ses arguments. Le MCP normalise précisément cette capacité, en offrant un cadre sûr et interopérable pour exposer et invoquer des outils — ici, la lecture d’un état ACS ou l’écriture d’un correctif.

2.12 Le format JSON-RPC

Les échanges MCP s'appuient sur **JSON-RPC 2.0**, un protocole d'appel de procédure à distance léger et sans état, où requêtes et réponses sont des objets JSON comportant une méthode, des paramètres et un identifiant de corrélation. Sa simplicité le rend adapté aussi bien au transport local par `stdio` qu'au transport réseau par HTTP.

2.13 Conteneurisation et orchestration

La **conteneurisation** (Docker) encapsule chaque service avec ses dépendances dans une unité isolée et reproductible. Les conteneurs partagent le noyau de l'hôte mais disposent d'espaces de noms distincts, garantissant isolation et légèreté. **Docker Compose** décrit, dans un fichier déclaratif, l'ensemble des services, leurs réseaux et leurs volumes, et les démarre de manière coordonnée. Cette approche assure la portabilité (déploiement identique en local ou dans le cloud) et la scalabilité horizontale visées par les besoins non fonctionnels.

2.14 Sécurité : hachage des identifiants

L'authentification des abonnés repose sur un code PIN dont seule l'empreinte est stockée, calculée avec **bcrypt**. Cet algorithme de hachage adaptatif intègre un *sel* aléatoire et un facteur de coût réglable, ce qui le rend résistant aux attaques par force brute et par tables précalculées. Combinée à l'exécution 100 % locale du LLM, cette mesure participe à la conformité et à la confidentialité recherchées.

2.15 Approfondissements théoriques

2.15.1 Pourquoi combiner ces briques plutôt qu'une seule

Une question légitime se pose : pourquoi assembler autant de technologies alors qu'un grand modèle de langage, à lui seul, semble déjà capable de « tout faire » ? La réponse tient à la nature du problème. Un LLM excelle à comprendre et à formuler du langage, mais il ne *sait* rien de fiable sur un équipement précis, et il ne peut pas *agir* sur le réseau. Le RAG, et plus encore le GraphRAG, comble le premier manque en lui fournissant une connaissance exacte et à jour. Le protocole d'orchestration d'outils comble le second en lui donnant des mains. Le standard de télégestion garantit que ces mains agissent sur de

véritables équipements. Chaque brique répond donc à une insuffisance précise des autres ; c’est leur *composition*, et non l’une d’elles isolément, qui produit un système à la fois intelligent, exact et capable d’agir.

Cette logique de composition est d’ailleurs caractéristique de l’ingénierie des systèmes d’IA modernes. On ne cherche plus le « modèle unique » qui résoudrait tout, mais l’*architecture* qui met le bon modèle au bon endroit, entouré des garde-fous nécessaires. Le modèle devient un composant parmi d’autres — central, certes, mais encadré. Cette vision, qui irrigue l’ensemble du présent travail, explique la place accordée à l’architecture logicielle autant qu’au modèle lui-même.

2.15.2 Une brève histoire des modèles de langage

Les approches du traitement du langage ont connu trois grandes vagues. Les méthodes *statistiques* (n-grammes) prédisaient un mot à partir des quelques précédents, sans réelle compréhension. Les réseaux *récurrents* (RNN, LSTM) ont introduit une mémoire séquentielle, mais peinaient sur les longues dépendances et se prêtaient mal à la parallélisation. L’arrivée du *Transformer*, en 2017, a levé ces deux verrous en traitant la séquence globalement grâce à l’attention. C’est cette architecture qui a rendu possibles les grands modèles actuels, entraînés sur des corpus massifs et capables de généraliser à des tâches non vues explicitement. Comprendre cette filiation aide à situer le choix d’un modèle comme Qwen 2.5 : héritier de cette lignée, il offre un compromis favorable entre capacité et empreinte, adapté à une exécution locale.

2.15.3 Le mécanisme d’attention, expliqué par l’exemple

L’attention peut se comprendre intuitivement comme une question posée à chaque mot : « à quels autres mots dois-je prêter attention pour t’interpréter ? ». Dans la phrase « la box ne s’allume plus, elle clignote rouge », le mot « elle » doit être relié à « box ». L’attention calcule, pour « elle », un poids élevé sur « box » et faible sur les autres mots, résolvant ainsi la référence. Répété sur de nombreuses têtes et couches, ce mécanisme capture des relations grammaticales, sémantiques et factuelles. Sa limite pratique est la *fenêtre de contexte* : au-delà d’un certain nombre de tokens, le modèle ne « voit » plus l’information. D’où l’intérêt d’une récupération ciblée qui n’injecte que l’essentiel — exactement ce que fait le GraphRAG.

2.15.4 Ingénierie de prompt et cadrage du modèle

La qualité des réponses d’un LLM dépend fortement de la manière dont on le sollicite. L’*ingénierie de prompt* consiste à cadrer le modèle : lui préciser son rôle (« tu es un assistant de support réseau »), lui fournir le contexte récupéré, et lui indiquer le format attendu. Dans une architecture agentique, ce cadrage inclut la description des outils disponibles et des règles d’usage. Un bon cadrage réduit la variabilité des réponses et canalise le raisonnement vers des décisions exploitables. Combiné à la récupération par graphe et à la liste blanche d’actions, il contribue à rendre le comportement de l’agent prévisible.

2.15.5 Bases orientées graphe contre bases relationnelles

Le choix de Neo4j plutôt qu’une base relationnelle classique mérite d’être justifié. Dans un modèle relationnel, exprimer « quel paramètre joue le rôle de VLAN pour ce modèle » suppose des jointures entre tables, coûteuses et peu lisibles dès que les relations se multiplient. Dans un modèle graphe, cette question devient un simple parcours d’arêtes, naturel et performant. Or notre problème est *intrinsèquement relationnel* : il s’agit de naviguer entre modèles, paramètres, rôles et dépendances. Le graphe épouse donc la structure du domaine, ce qui simplifie autant l’expression des requêtes que la maintenance du modèle.

2.15.6 Le protocole CWMP en détail

Le TR-069/CWMP organise le dialogue entre l’ACS et le CPE autour de sessions initiées par l’équipement, qui envoie périodiquement un message d’*Inform*. L’ACS répond alors par une suite d’appels de procédure : lecture (*GetParameterValues*), écriture (*SetParameterValues*), ajout ou suppression d’objets, redémarrage. Les paramètres sont désignés par des chemins hiérarchiques au sein d’un modèle de données normalisé, que chaque constructeur étend par des nœuds propriétaires — précisément la source de la variabilité que notre graphe absorbe. Cette standardisation, quoique partielle, est ce qui rend envisageable une automatisation multi-constructeurs : sans elle, chaque équipement exigerait un pilote spécifique.

2.15.7 La sûreté des agents autonomes

Confier une capacité d’action à un système fondé sur un LLM soulève une question de sûreté légitime. La réponse retenue dans ce projet repose sur trois principes complémentaires. Le premier est la *restriction du champ d’action* : l’agent ne peut invoquer qu’un ensemble fini d’outils, et n’écrire que des rôles éditables. Le deuxième est la *traçabilité* : chaque décision est journalisée avec les faits qui l’ont motivée. Le troisième est la *vérification a posteriori* : aucune action n’est réputée réussie sans constat de son effet. Ensemble, ces principes transforment un agent potentiellement imprévisible en un exécutant borné et contrôlable, condition nécessaire à tout déploiement en production.

2.16 Illustrations et exemples des concepts

Afin de rendre concrets les fondements présentés ci-dessus, cette section les illustre par des exemples directement tirés du domaine du support ISP.

2.16.1 Exemple : de la phrase de l’abonné aux tokens

Lorsqu’un abonné écrit « Mon Wi-Fi est coupé depuis ce matin », le modèle ne « lit » pas la phrase comme un humain : il la découpe en tokens, puis en vecteurs. Le découpage produit approximativement la suite [Mon] [Wi] [-] [Fi] [est] [coup] [é] [depuis] [ce] [matin]. Chaque token est projeté dans un espace vectoriel où « Wi-Fi », « connexion » et « réseau » sont proches. C’est cette proximité sémantique qui permet ensuite à l’agent de rattacher la plainte à la catégorie technique « perte de connectivité WAN », même si l’abonné n’emploie aucun terme technique. Ce point est plus profond qu’il n’y paraît. Dans un système fondé sur des règles, il faudrait prévoir explicitement chaque tournure possible ; le nombre de cas explose et la maintenance devient intenable. Le recours aux plongements déplace le problème d’une énumération exhaustive vers une *généralisation* apprise, bien plus robuste face à la créativité du langage humain. C’est l’une des raisons pour lesquelles les LLM ont supplanté les anciens agents conversationnels à base de scripts.

Exemple

Robustesse au langage familier. Les formulations « j'ai plus internet », « ça marche pas », « la box clignote rouge » sont toutes rapprochées, dans l'espace des plongements, d'un même noyau sémantique. L'agent n'a donc pas besoin d'un catalogue exhaustif de phrases-types : il généralise à partir du sens, ce qui améliore nettement le taux de compréhension par rapport à un arbre de décision rigide.

2.16.2 Exemple : hallucination sans RAG, ancrage avec GraphRAG

Considérons la question interne : « Quel est le chemin TR-069 du VLAN WAN pour un ONT Huawei HG8145 ? ». Un LLM interrogé seul, sans source, peut produire une réponse plausible mais fausse — par exemple inventer un chemin générique `...WANConnection.VLAN` qui n'existe pas chez ce constructeur. C'est une *hallucination*. Avec le GraphRAG, la réponse est d'abord *récupérée* dans le graphe : le parcours part du nœud du modèle Huawei, suit la relation portant le rôle `vlan_id` et renvoie le chemin réel se terminant par `X_HW_VLAN`. La réponse est alors ancrée dans un fait vérifiable et non plus devinée.

Exemple

Pourquoi c'est décisif ici. Une hallucination sur un chemin TR-069 n'est pas une simple erreur de style : elle conduirait le moteur de remédiation à écrire au mauvais endroit, donc à échouer silencieusement, voire à modifier un paramètre non voulu. L'ancrage par le graphe est ce qui rend l'action *sûre*.

2.16.3 Exemple : un parcours GraphRAG pas à pas

Reprenons la plainte « pas d'Internet » sur un modèle Nokia. Le raisonnement suit les relations du graphe plutôt qu'une recherche de mots-clés :

1. point d'entrée : le nœud `(:Router {id:"NOKIA_1425"})` ;
2. parcours des relations `HAS_PARAM` pertinentes pour la connectivité : `ppp_connection_status`, puis `vlan_id` ;
3. récupération des `(:Parameter)` cibles, avec leurs chemins réels (ici, suffixe `VLANIDMark` propre à Nokia) et l'attribut `editable` ;
4. restitution d'un contexte compact et exact, injecté dans le raisonnement de l'agent.

Le graphe a ainsi préservé la *dépendance* entre l'état PPP et la configuration VLAN, là où un simple index vectoriel aurait pu retourner deux fragments sans lien.

2.16.4 Exemple : un échange MCP en JSON-RPC

La découverte et l'exécution d'outils via MCP se traduisent par des messages JSON-RPC. À titre illustratif, la découverte des outils exposés par le serveur RaDuce puis l'appel du correctif VLAN prennent la forme suivante :

```

1 --> {"jsonrpc":"2.0","id":1,"method":"tools/list"}
2 <-- {"jsonrpc":"2.0","id":1,"result":{"tools":[
3     {"name":"set_vlan","description":"Corrige le VLAN WAN"},
4     {"name":"restart_ppp","description":"Relance la session PPP"} ]}}
5
6 --> {"jsonrpc":"2.0","id":2,"method":"tools/call",
7     "params":{"name":"set_vlan","arguments":{"serial":"SN-NK-2007","vlan_id"
8     ↪ :100}}}
9 <-- {"jsonrpc":"2.0","id":2,"result":{"status":"UP","path":"...VLANIDMark",
10    ↪ "written":100}}
```

Listing 2.2. Découverte puis appel d'un outil via MCP (JSON-RPC, illustratif).

On voit que l'agent n'a pas eu à connaître le chemin exact : il a demandé une *intention* (« corrige le VLAN ») et la couche d'action, adossée au graphe, a résolu le chemin propre à Nokia.

2.16.5 Exemple : un échange TR-069 (lecture puis écriture)

Au niveau du protocole de télégestion, la lecture puis la correction d'un paramètre correspondent aux méthodes normalisées *GetParameterValues* et *SetParameterValues* :

```

1 GetParameterValues( "...WANPPPConnection.1.X_HW_VLAN" )
2 => 999                                     # VLAN incorrect detecte
3
4 SetParameterValues( "...WANPPPConnection.1.X_HW_VLAN" = 100 )
5 => 0 (SUCCESS)                             # correctif applique au bon chemin
```

Listing 2.3. Lecture puis écriture d'un paramètre TR-069 (illustratif).

Cet enchaînement, invisible pour l'abonné, constitue l'action réelle qui distingue notre solution d'un simple chatbot documentaire.

2.17 Conclusion

Ce chapitre a présenté les briques technologiques mobilisées par la plateforme. Leur combinaison — LLM local, GraphRAG sur Neo4j, couche d’action MCP/UTCP et télégestion TR-069 — constitue l’originalité de la solution. Le chapitre suivant analyse les besoins et la méthodologie de conduite du projet.

CHAPITRE 3

Analyse et spécification des besoins

3.1 Introduction

Ce chapitre présente la méthodologie de conduite de projet retenue, identifie les acteurs du système, spécifie les besoins fonctionnels et non fonctionnels, puis les formalise à l'aide d'un diagramme de cas d'utilisation et du raffinement de quelques cas représentatifs.

3.2 Méthodologie de conduite de projet

Deux cadres de gestion agiles ont été comparés : **Kanban** et **2TUP**. Kanban organise le travail en flux continu sur un tableau (*À faire* → *En cours* → *Fait*) avec une limite du travail en cours ; il est léger mais faible sur la modélisation et l'architecture en amont. Le **2TUP** (Two Track Unified Process) est un processus itératif piloté par UML, structuré en deux branches — fonctionnelle et technique — qui convergent, et produisant naturellement les diagrammes attendus (cas d'usage, classes, séquence, déploiement).

Choix : 2TUP. Le projet étant fortement centré sur l'architecture et la modélisation, avec une réelle double piste fonctionnelle et technique, le **2TUP** a été retenu comme processus de développement. Un tableau Kanban a été utilisé en complément pour le suivi opérationnel des tâches.

Le processus 2TUP tire sa force de sa forme en Y. La **branche de gauche**, dite fonctionnelle, part des besoins des utilisateurs et les affine sans se préoccuper de la technologie : elle produit les cas d'utilisation, les scénarios et le modèle du domaine. La **branche de droite**, dite technique, explore en parallèle les contraintes et les choix de la plateforme : langages, frameworks, protocoles, infrastructure. Ces deux branches, menées indépendamment, **convergent** ensuite dans une phase de conception qui réconcilie le « quoi » et le « comment » pour aboutir à la réalisation. Cette organisation évite deux écueils symétriques : concevoir une belle architecture qui ne répond pas aux besoins réels, ou répondre aux besoins par une architecture fragile et non pérenne.

Appliqué à notre projet, ce découpage s'est révélé particulièrement pertinent. La branche fonctionnelle a permis de stabiliser tôt le parcours de support — décrire, diagnostiquer, corriger, vérifier, escalader — tandis que la branche technique explorait les briques les plus risquées (graphe de connaissances, orchestration MCP, télégestion). La convergence a alors consisté à faire du graphe le point de rencontre des deux branches : porteur du sens métier (les rôles) autant que de la réalité technique (les chemins).

Le 2TUP a été appliqué comme suit. La **branche fonctionnelle** a traité le portail client et le flux de support (diagnostiquer → corriger → vérifier → escalader), les cas d'usage et les besoins. La **branche technique** a traité les microservices Docker, le graphe Neo4j, les serveurs MCP (Genie/RaDuce), MongoDB et la résolution rôle → chemin TR-069. Les deux branches ont convergé au fil de quatre itérations incrémentales (tableau 3.1).

Table 3.1. Itérations du développement selon le 2TUP.

Itération	Livraison incrémentale
Itération 1	Portail client et simulateur d'état sur MongoDB.
Itération 2	Graphe de connaissances Neo4j et agent GraphRAG.
Itération 3	Serveurs MCP (Genie / RaDuce) et exécution UTCP « code mode ».
Itération 4	Graphe rendu déterminant : résolution rôle → chemin, statut dérivé, escalade réelle.

3.3 Identification des acteurs

Le système fait intervenir trois acteurs principaux :

- **Le client (abonné)** : il s'authentifie, décrit un problème en langage naturel et suit ses tickets.
- **L'agent IA / système** : acteur logiciel autonome qui diagnostique la panne, interroge le graphe et exécute les correctifs. Modéliser l'IA comme un acteur à part entière, et non comme un simple composant interne, est un choix significatif. Il reconnaît que l'agent perçoit, décide et agit avec une forme d'autonomie, ce qui conduit à définir explicitement ce qu'il est autorisé à faire, dans quelles limites et sous quelle supervision — précisément le cadrage qu'exige le déploiement responsable d'un système autonome.
- **L'ingénieur NOC** : il supervise l'état du parc, visualise le raisonnement de l'IA et peut injecter ou corriger des pannes à des fins de test.

3.4 Spécification des besoins fonctionnels

Les besoins fonctionnels sont organisés par acteur dans le tableau 3.2.

Table 3.2. Besoins fonctionnels par acteur.

Client	Agent IA / Système	Ingénieur NOC
S'authentifier (téléphone + PIN)	Diagnostiquer la panne du routeur	Superviser l'état du parc
Décrire un problème en langage naturel	Interroger le graphe via GraphRAG	Injecter / corriger une panne (test)
Suivre ses tickets en cours	Exécuter les outils MCP (correctifs)	Visualiser le raisonnement de l'IA
Consulter l'historique des réparations	Créer un ticket et escalader	Gérer et rechercher les tickets

3.5 Spécification des besoins non fonctionnels

Au-delà des fonctionnalités, le système doit satisfaire un ensemble d'exigences de qualité (tableau 3.3).

Table 3.3. Besoins non fonctionnels.

Exigence	Description
Performance	Faible latence ; réduction forte du MTTR.
Sécurité	Authentification par PIN (haché avec bcrypt), outils MCP validés, IA 100 % locale.
Scalabilité	Microservices indépendants, montée en charge horizontale.
Disponibilité	Triage automatisé 24/7, sans file d'attente.
Maintenabilité	Code modulaire, conteneurs isolés, faible couplage.
Portabilité	Déploiement identique en local ou dans le cloud.

Ces exigences ne sont pas indépendantes : elles se renforcent mutuellement. Le choix des microservices sert à la fois la scalabilité, la maintenabilité et la portabilité. L'exécution locale du LLM sert la sécurité et la maîtrise des coûts. La conception « sûre par construction » de la couche d'action sert la fiabilité sans nuire à la performance. Cette cohérence entre les besoins non fonctionnels et les choix d'architecture est un indicateur de la solidité de la conception : chaque décision technique répond à plusieurs exigences à la fois, plutôt que d'en satisfaire une au détriment des autres.

3.6 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation global (figure 3.1) synthétise les interactions entre les acteurs et le système. Le client déclenche, via le cas « Décrire un problème », le pipeline automatique de diagnostic et de correction (relation «include»), tandis que l'ingénieur NOC supervise le parc et peut injecter ou corriger des pannes.

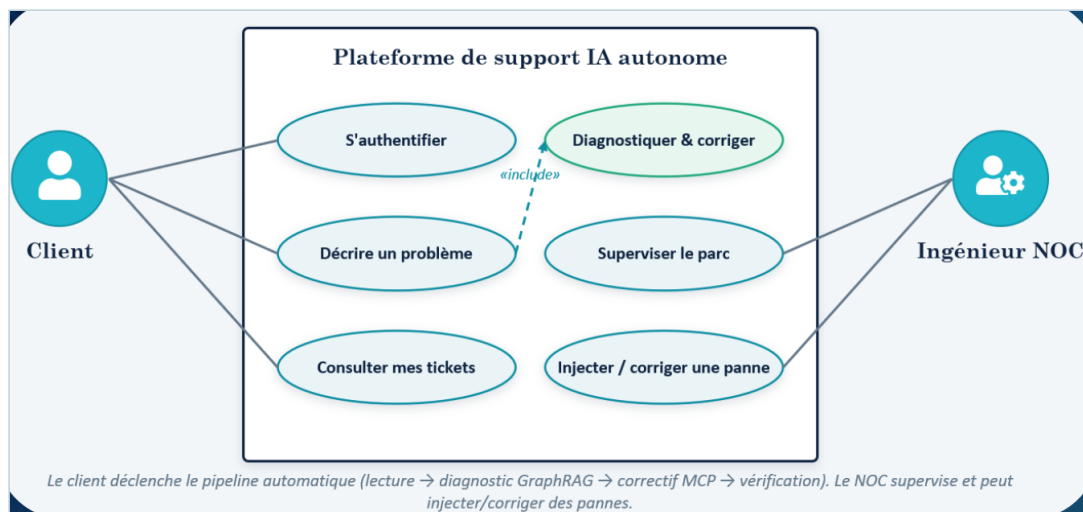


Figure 3.1. Diagramme de cas d'utilisation de la plateforme.

3.6.1 Raffinement de cas d'utilisation

Le tableau 3.4 décrit textuellement le cas d'utilisation central « Diagnostiquer & corriger une panne ».

Table 3.4. Description du cas d'utilisation « Diagnostiquer & corriger une panne ».

Rubrique	Contenu
Acteur principal	Agent IA / Système (déclenché par le client)
Pré-condition	Le client est authentifié et a décrit un problème.
Scénario nominal	(1) Lire l'état du routeur via GenieACS ; (2) résoudre les rôles → chemins TR-069 et diagnostiquer via GraphRAG ; (3) appeler l'outil correctif RaDuce approprié ; (4) vérifier le statut dérivé ; (5) confirmer la résolution et clôturer le ticket.
Scénario alternatif	Si la panne n'est pas réparable à distance (signal optique faible, défaut matériel) ou si la vérification échoue, le système crée un ticket et escalade vers un technicien.
Post-condition	Le routeur est de nouveau opérationnel, ou un ticket d'escalade est ouvert.

3.6.2 Cas d'utilisation « S'authentifier »

Table 3.5. Description du cas d'utilisation « S'authentifier ».

Acteur principal	Client (abonné)
Pré-condition	L'abonné dispose d'un compte et d'un code PIN.
Scénario nominal	(1) L'abonné saisit son numéro de téléphone et son PIN ; (2) le système vérifie l'empreinte bcrypt ; (3) une vérification par e-mail confirme l'identité ; (4) l'accès au portail est accordé.
Scénario alternatif	PIN erroné : le système refuse l'accès et journalise la tentative.
Post-condition	Une session authentifiée est ouverte.

3.6.3 Cas d'utilisation « Injecter / corriger une panne »

Table 3.6. Description du cas d'utilisation « Injecter / corriger une panne » (NOC).

Acteur principal	Ingénieur NOC
Pré-condition	L'ingénieur est authentifié sur le tableau de bord.
Scénario nominal	(1) L'ingénieur sélectionne un routeur ; (2) il choisit un type de panne à injecter ; (3) le système écrit l'état correspondant (par chemins TR-069) et le routeur passe en panne ; (4) l'ingénieur peut déclencher la remédiation et observer le retour à l'état opérationnel.
Post-condition	L'état du routeur reflète la panne injectée puis, le cas échéant, sa correction.

3.6.4 Cas d'utilisation « Décrire un problème »

3.6.5 Cas d'utilisation « Superviser le parc »

3.6.6 Vue d'ensemble du flux fonctionnel

Les besoins fonctionnels s'articulent en un flux unique : l'abonné décrit → l'agent diagnostique et corrige → le système vérifie et notifie → le NOC supervise et arbitre les escalades. Cette linéarité apparente masque une coordination entre plusieurs services,

Table 3.7. Description du cas d'utilisation « Décrire un problème ».

Acteur principal	Client (abonné)
Pré-condition	L'abonné est authentifié.
Scénario nominal	(1) L'abonné saisit sa plainte en langage naturel ; (2) l'agent interprète l'intention ; (3) le pipeline de diagnostic et de correction est déclenché (relation «include») ; (4) l'abonné reçoit une réponse et, le cas échéant, la confirmation de résolution.
Scénario alternatif	Si la demande sort du périmètre technique, l'agent oriente l'abonné ou crée un ticket.
Post-condition	Une réponse est fournie ; un ticket est éventuellement ouvert.

Table 3.8. Description du cas d'utilisation « Superviser le parc » (NOC).

Acteur principal	Ingénieur NOC
Pré-condition	L'ingénieur est authentifié sur le tableau de bord.
Scénario nominal	(1) L'ingénieur visualise l'état du parc en temps réel ; (2) il filtre les équipements en panne ; (3) il consulte le raisonnement de l'IA sur un incident ; (4) il suit la résolution ou intervient si nécessaire.
Post-condition	L'état du parc est supervisé et les incidents sont tracés.

détaillée dans la conception ; du point de vue de l'utilisateur, elle se traduit par une expérience simple et immédiate.

3.7 Planification et gestion des risques

3.7.1 Planification du projet

Le projet a été conduit en quatre itérations alignées sur les branches du 2TUP (tableau 3.1). Chaque itération a livré un incrément fonctionnel testable, réduisant le risque d'intégration en fin de parcours. Le tableau 3.9 récapitule les phases et leurs livrables principaux.

Table 3.9. Phases du projet et livrables.

Phase	Livrable principal
Étude et cadrage	Analyse de l'existant, problématique, choix technologiques.
Analyse et conception	Besoins, diagrammes UML, architectures, modèles de données.
Développement itératif	Portail, graphe, serveurs MCP, boucle de remédiation.
Intégration et tests	Validation de bout en bout sur le banc à douze routeurs.
Documentation	Rédaction du présent rapport et des supports.

3.7.2 Analyse des risques

Plusieurs risques ont été identifiés et anticipés (tableau 3.10).

3.8 Conclusion

Ce chapitre a formalisé la méthodologie 2TUP, les acteurs et les besoins fonctionnels et non fonctionnels du système, illustrés par un diagramme de cas d'utilisation. Ces spécifications servent de base à la conception détaillée présentée au chapitre suivant.

Table 3.10. Principaux risques et mesures d'atténuation.

Risque	Mesure d'atténuation
Hallucination du LLM sur un chemin technique	Ancrage systématique par le graphe (GraphRAG).
Action erronée sur un paramètre sensible	Liste blanche des rôles éditables.
Indisponibilité du graphe	Repli automatique sur une carte de chemins en cache.
Fausse résolution	Statut dérivé recalculé après chaque correctif.
Diversité des constructeurs	Résolution rôle → chemin externalisée dans le graphe.

CHAPITRE 4

Conception

4.1 Introduction

Ce chapitre détaille la conception de la plateforme. Il présente successivement l'architecture globale, l'architecture logique en couches, l'architecture physique en microservices, le diagramme de classes, les modèles de données (MongoDB et graphe Neo4j), le diagramme de séquence de résolution d'une panne, et enfin la conception de la résolution rôle → chemin TR-069, pierre angulaire du système.

4.2 Architecture globale

L'architecture globale (figure 4.1) organise le système en couches fonctionnelles. La couche de **présentation** regroupe le portail client et le tableau de bord NOC. La couche d'**intelligence et décision** héberge le moteur de décision (agent) et le LLM local (Ollama / Qwen 2.5). La couche de **connaissance et action** réunit la lecture de l'état via GenieACS, le diagnostic GraphRAG via neo4j-agent, et l'écriture des correctifs via RaDuce. Enfin, la couche de **données** centralise l'état des routeurs dans MongoDB, indexé par les chemins TR-069. Ce découpage en couches n'est pas qu'une commodité de présentation : il traduit un principe de conception, la *séparation des préoccupations*. Chaque couche ne connaît de la suivante que le service qu'elle lui rend, jamais ses détails internes. La couche de présentation ignore comment un correctif est appliqué ; la couche d'action ignore comment l'abonné a formulé sa plainte. Cette étanchéité rend le système compréhensible, testable couche par couche, et surtout évolutif : on peut remplacer le LLM, changer d'ACS ou faire évoluer l'interface sans propager le changement à l'ensemble. C'est la traduction concrète, dans l'architecture, des exigences de maintenabilité et de faible couplage du cahier des charges. Cette séparation se lit aussi dans le trajet des données. Une information descend des couches hautes vers les basses lorsqu'il s'agit d'agir — une intention devient un rôle, le rôle un chemin, le chemin une valeur écrite — puis remonte lorsqu'il s'agit de constater : un état lu devient un statut affiché. À chaque frontière, l'information change de niveau d'abstraction. Comprendre l'architecture, c'est donc la voir non comme un empilement statique de boîtes, mais comme un flux vivant reliant la parole de l'abonné à l'état réel de son équipement.

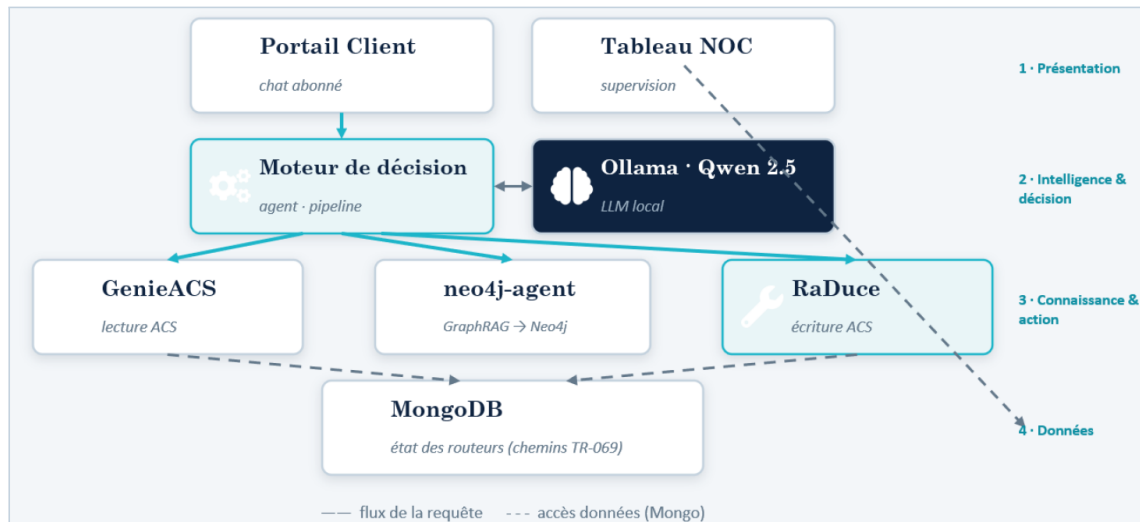


Figure 4.1. Architecture globale de la plateforme.

4.3 Architecture logique en couches

La vue logique (figure 4.2) précise la répartition des responsabilités entre les couches et les flux d'information qui les traversent, depuis la requête de l'abonné jusqu'à l'action sur l'équipement et le retour de confirmation.

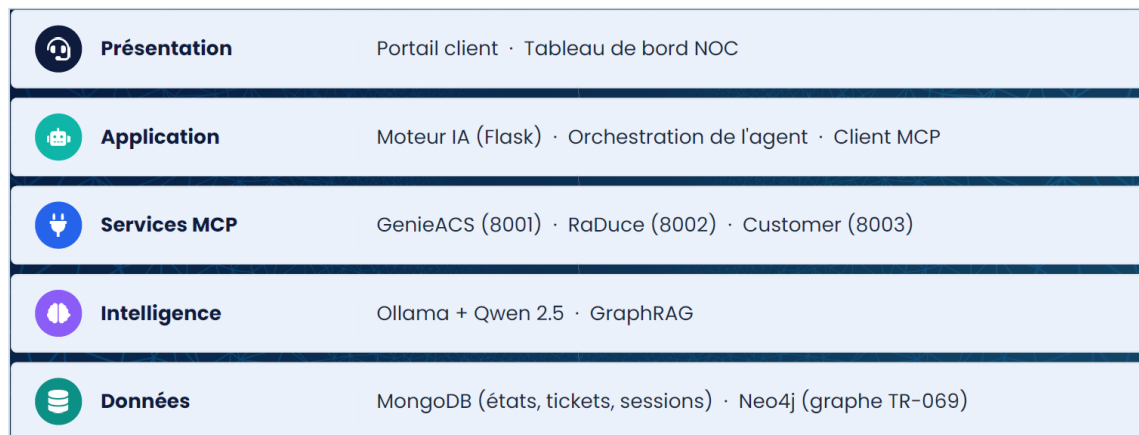


Figure 4.2. Architecture logique en couches.

4.4 Architecture physique : microservices Docker

La plateforme est déployée sous forme de **microservices conteneurisés** orchestrés par Docker Compose (figures 4.3 et 4.4). Chaque composant fonctionnel est isolé dans son propre conteneur, communiquant via un réseau bridge dédié (**mypfe-net**). Cette organisation garantit l'isolation, la scalabilité horizontale et un déploiement identique en local ou dans le cloud. Le choix des microservices, plutôt qu'une application monolithique,

se justifie par la diversité des technologies mises en jeu : un service d'inférence LLM, une base graphe, une base documentaire, des serveurs d'outils et une interface web n'ont ni les mêmes cycles de vie, ni les mêmes besoins en ressources. Les isoler permet de les développer, de les déployer et de les faire monter en charge indépendamment. Cette modularité a également un bénéfice pédagogique et opérationnel : une panne dans un service (par exemple l'indisponibilité du graphe) n'effondre pas l'ensemble, et le système sait alors basculer sur un mode dégradé maîtrisé.



Figure 4.3. Architecture physique — conteneurs Docker.



Figure 4.4. Vue microservices du déploiement.

Le tableau 4.1 recense les principaux services et leurs rôles.

4.5 Diagramme de classes

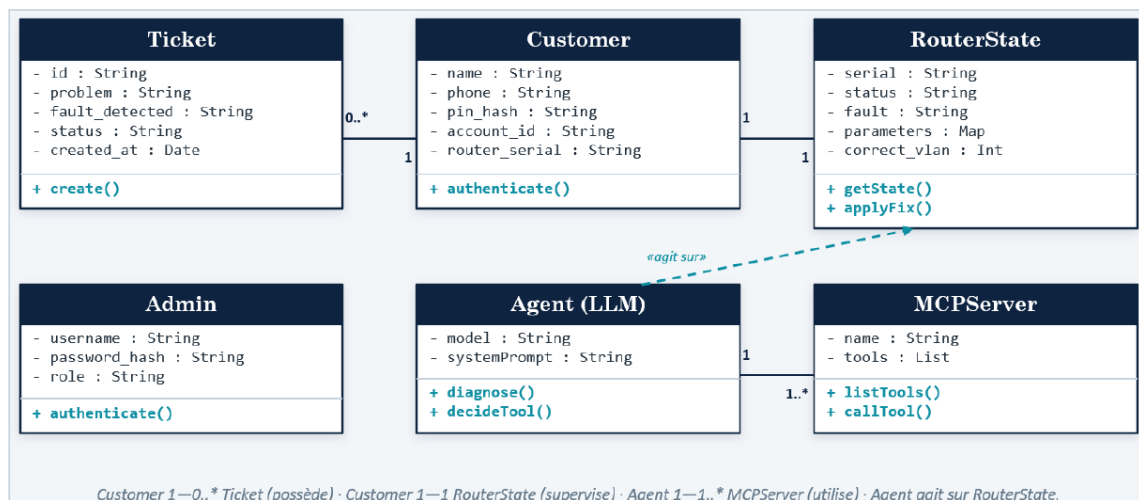
Le diagramme de classes (figure 4.5) modélise les entités métier et leurs relations. La classe **Customer** possède ses **Ticket** ($1 \rightarrow 0..*$) et supervise un **RouterState** ($1 \rightarrow 1$). L'**Agent** orchestre un ou plusieurs **MCPServer** ($1 \rightarrow 1..*$) et agit sur le **RouterState**. La classe **Admin** gère l'authentification des opérateurs NOC.

Chaque classe porte une responsabilité claire. La classe **Customer** représente l'abonné : elle détient son identité (nom, téléphone), l'empreinte de son code PIN et le lien vers son

Table 4.1. Microservices de la plateforme.

Service	Port	Rôle
mypfe-app	5000	Portail client et moteur de décision (agent).
neo4j-agent	8000	Couche GraphRAG : interrogation du graphe, résolution rôle → chemin.
genie	8001	ACS simulé (TR-069) : lecture de l'état des routeurs.
raduce	8002	Moteur de remédiation : écriture des correctifs par chemin.
customer	8003	Service de gestion des profils abonnés.
code-mode	node	Exécution UTCP « code mode » des routines d'orchestration.
ollama	11434	LLM local (Qwen 2.5).
mongodb	27017	Persistance de l'état des routeurs et des tickets.

équipement ; sa méthode `authenticate()` valide l'accès. La classe `Ticket` matérialise un incident : elle en conserve la description, la panne détectée, le statut et l'horodatage, permettant le suivi et l'historique. La classe `RouterState` est le cœur technique : elle expose l'état d'un routeur — son numéro de série, son statut dérivé, la panne courante, ses paramètres indexés par chemin et la valeur VLAN attendue — et offre les opérations `getState()` et `applyFix()`. La classe `Agent` incarne l'intelligence : dotée d'un modèle et d'un prompt système, elle `diagnose()` et `decideTool()` pour choisir l'action. La classe `MCPServer` abstrait un serveur d'outils : elle sait `listTools()` et `callTool()`, masquant au reste du système les détails du protocole. Enfin, la classe `Admin` gère les opérateurs NOC. Cette répartition respecte le principe de responsabilité unique : chaque classe a une raison d'exister et une seule.

**Figure 4.5.** Diagramme de classes de la plateforme.

4.6 Modèles de données

4.6.1 Modèle documentaire MongoDB

L'état de chaque routeur est persisté dans une collection MongoDB. Point de conception essentiel : le dictionnaire `parameters` est **indexé par les chemins TR-069 réels**, à l'image de l'arborescence d'un véritable équipement ACS. Le statut (UP/DOWN) n'est pas un simple drapeau mais une valeur **dérivée** des paramètres : un routeur est considéré comme opérationnel si et seulement si aucun diagnostic ne détecte de panne. Ce choix de modélisation, apparemment mineur, a des conséquences profondes. En refusant de stocker le statut comme un simple champ que l'on pourrait mettre à jour « à la main », on élimine toute possibilité d'incohérence entre l'état affiché et l'état réel. Le statut devient une *conséquence* des faits et non une *déclaration* : il ne peut mentir. C'est cette même philosophie qui, appliquée à la boucle de remédiation, garantit qu'une correction n'est validée que si elle produit réellement l'effet attendu.

4.6.2 Graphe de connaissances Neo4j

Le graphe (figure 4.6) modélise la documentation technique. Un nœud `:Router` (portant `id`, `vendor`, `technology`) est relié à des nœuds `:Parameter` (portant `path`, `label`, `editable`) par une relation `:HAS_PARAM`. Cette relation porte une propriété déterminante, le **rôle canonique** (par exemple `vlan_id`), qui rend la résolution déterministe. Le graphe compte douze routeurs réels et environ sept-cents relations `HAS_PARAM`.

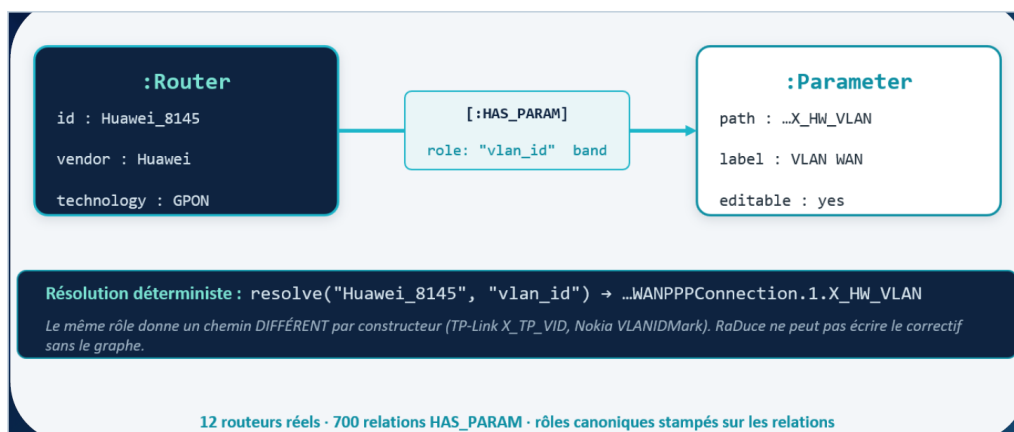


Figure 4.6. Schéma du graphe de connaissances Neo4j : `(:Router)-[:HAS_PARAM {role}]->(:Parameter)`.

4.7 Diagramme de séquence : résolution d'une panne

Le diagramme de séquence (figure 4.7) décrit le déroulement complet d'une résolution. Le portail interroge GenieACS pour lire l'état (par chemins TR-069), résout le rôle en chemin et diagnostique via Neo4j, appelle RaDuce pour écrire le correctif au bon chemin (*SetParameterValues*), puis vérifie le statut dérivé avant de confirmer la résolution à l'abonné.

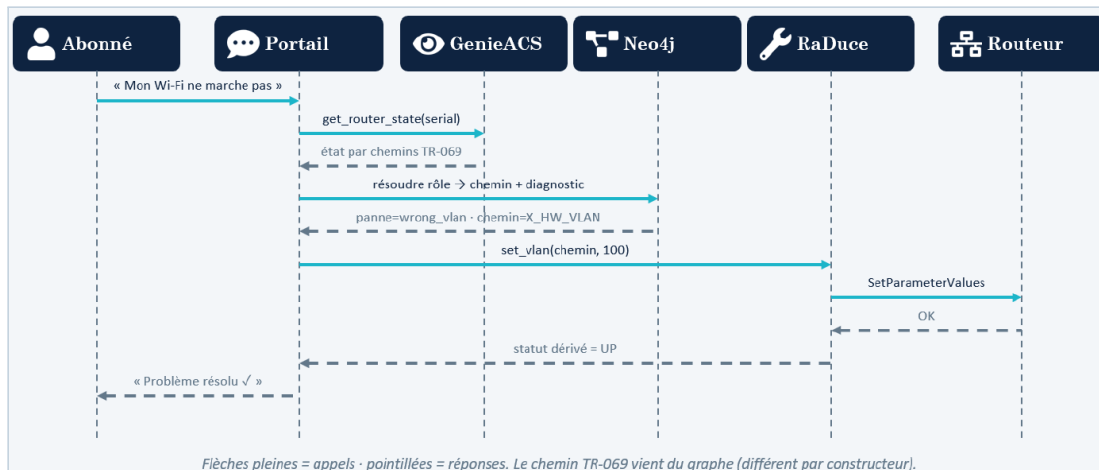


Figure 4.7. Diagramme de séquence de la résolution d'une panne.

4.8 Scénarios de séquence complémentaires

Au-delà de la résolution d'une panne, deux autres scénarios structurants méritent d'être décrits.

4.8.1 Séquence d'authentification

Lorsqu'un abonné se connecte, le portail transmet le couple (téléphone, PIN) au service client, qui compare le PIN à son empreinte bcrypt. En cas de succès, une vérification par e-mail est déclenchée : un code est envoyé puis validé, ouvrant une session authentifiée. Ce double contrôle protège l'accès sans imposer de mot de passe complexe à retenir.

4.8.2 Séquence d'injection de panne (NOC)

Pour tester le système, l'ingénieur NOC sélectionne un routeur et un type de panne. Le service GenieACS écrit alors, aux chemins TR-069 appropriés, les valeurs correspondant à la panne choisie ; le statut dérivé bascule à DOWN. L'ingénieur peut ensuite déclencher la

remédiation et observer, étape par étape, le retour à l'état opérationnel. Ce mécanisme fait du tableau NOC un véritable banc d'essai intégré.

4.8.3 Construction et alimentation du graphe

Le graphe est construit en deux temps. D'abord, les modèles de routeurs et leurs paramètres sont importés sous forme de nœuds et de relations. Ensuite, une étape d'*estampillage* parcourt les relations pour y inscrire le rôle canonique et compléter les chemins standards manquants. Cette séparation permet d'enrichir la connaissance sans reconstruire le graphe : ajouter un modèle revient à insérer quelques nœuds et relations correctement étiquetés.

4.9 Conception de la résolution rôle $\rightarrow$ chemin TR-069

La contribution centrale du projet est une **couche de résolution** qui découple le paramètre *logique* manipulé par l'agent du *chemin* TR-069 réel, spécifique à chaque constructeur. À un rôle canonique (par exemple `vlan_id`) correspond, pour chaque modèle, un chemin différent (Huawei `X_HW_VLAN`, TP-Link `X_TP_VID`, Nokia `VLANIDMark`). C'est le graphe qui connaît cette correspondance : sans lui, le moteur de remédiation ne pourrait pas écrire le correctif. La résolution s'effectue par une requête Cypher sur la relation portant le rôle (listing 4.1).

```

1 MATCH (r:Router)-[rel:HAS_PARAM {role: $role}]->(p:Parameter)
2 WHERE toLower(r.id) CONTAINS toLower($model)
3 RETURN p.path AS path, p.editable AS editable
4 LIMIT 1

```

Listing 4.1. Résolution du chemin TR-069 à partir d'un rôle (Cypher).

Statut dérivé et boucle de sûreté. Parce que le statut est recalculé à partir des paramètres après chaque correctif, un correctif qui n'élimine pas la cause racine laisse le routeur en panne et déclenche l'escalade. Aucune action ne peut donc « réussir faussement », ce qui rend la branche d'escalade réellement vivante.

4.10 Catalogue des rôles et modèle de pannes

La couche de résolution repose sur un ensemble fini de **rôles canoniques**, séparés en rôles de *lecture* (interrogés lors du diagnostic) et rôles d'*écriture* (seuls autorisés à être modifiés). Le tableau 4.2 en donne la liste.

Table 4.2. Catalogue des rôles canoniques.

Type	Rôle	Signification
Lecture	ppp_ connection_ status	État de la connexion PPP (Connected / Disconnected / Unconfigured).
Lecture	last_ connection_ error	Dernière erreur de connexion (authentification, délai).
Lecture	vlan_id	Identifiant du VLAN WAN.
Lecture	dns_servers	Serveurs DNS configurés.
Lecture	rx_ optical_ power	Puissance optique reçue (GPON), en dBm.
Lecture	dsl_crc_ errors	Compteur d'erreurs CRC (xDSL).
Lecture	uptime	Durée de fonctionnement de la session.
Écriture	vlan_id	Correction du VLAN WAN.
Écriture	ppp_password	Re-poussée des identifiants (redémarrage PPP).
Écriture	dns_servers	Poussée de serveurs DNS valides.
Écriture	reboot	Redémarrage à distance.

À chaque panne correspond un correctif déterministe (tableau 4.3). Les pannes physiques ne disposent d'aucun correctif à distance et sont directement escaladées.

4.11 Résolution par constructeur : preuve de la nécessité du graphe

Le tableau 4.4 illustre, pour les douze modèles de routeurs du banc d'essai, le chemin TR-069 réel associé au rôle `vlan_id`. Un **même rôle logique** se résout en **trois familles de chemins distinctes** selon le constructeur, ce qui démontre qu'aucune approche par simple dictionnaire figé ne pourrait remplacer le graphe : c'est bien la connaissance modélisée qui permet à RaDuce d'écrire au bon endroit.

Table 4.3. Modèle de pannes et correctifs associés.

Panne	Condition de détection	Correctif
ppp_auth_failure	PPP déconnecté + erreur d'authentification	restart_ppp
random_disconnect	PPP déconnecté (délai serveur)	restart_ppp
wrong_vlan	VLAN $\notin \{0, 100\}$ / non configuré	set_vlan
dns_failure	DNS = 0.0.0.0	set_dns
weak_signal	rx_optical_power < -27 dBm	escalade technicien
hardware_fault	dsl_crc_errors > 100	escalade technicien

Table 4.4. Résolution du rôle `vlan_id` en chemin TR-069 pour les douze modèles.

Modèle	Technologie	Suffixe du chemin VLAN
Huawei_8145	GPON	X_HW_VLAN
Huawei_HG8245H5	GPON	X_HW_VLAN
HG8145X7-10	GPON	X_HW_VLAN
Huawei_V163	GPON	X_HW_VLAN
V166a-20	GPON	X_HW_VLAN
NOKIA_1425	GPON	VLANIDMark
NOKIA_2425	GPON	VLANIDMark
NOKIA_G-240W-A	GPON	VLANIDMark
NOKIA_G-240W-F	GPON	VLANIDMark
G-1426G-D	GPON	VLANIDMark
D-Link_DSL224	xDSL	VLANIDMark
TP-LINK_VC220-G3v	xDSL	X_TP_VID

4.12 Schéma documentaire MongoDB

Le listing 4.2 présente la structure d'un document `router_states` tel que persisté dans MongoDB. Le dictionnaire `parameters` est indexé par les chemins TR-069 réels, et le champ `status` est recalculé à partir de ce dictionnaire.

```

1 {
2   "serial": "SN-HW-1001",
3   "account_id": "ACC1001",
4   "model": "Huawei_8145",
5   "status": "DOWN",           // derive: UP ssi aucun diagnostic ne detecte
                                ⇨ de panne
6   "fault": "wrong_vlan",
7   "correct_vlan": 100,
8   "parameters": {
9     "InternetGatewayDevice.WANDevice.1.WANConnectionDevice.1.WANPPPConnection.1.
                                ⇨ ConnectionStatus": "Unconfigured",
10    "InternetGatewayDevice.WANDevice.1.WANConnectionDevice.1.WANPPPConnection.1.
                                ⇨ X_HW_VLAN": 999
11  }
12 }
```

Listing 4.2. Structure d'un document d'état de routeur (MongoDB).

4.13 Exemples détaillés de résolution

Cette section déroule, sur des cas concrets, le mécanisme de résolution rôle → chemin et le calcul du statut dérivé, afin d'en montrer le fonctionnement pas à pas.

4.13.1 Exemple : un même correctif, trois chemins

Supposons que trois abonnés équipés respectivement d'un ONT Huawei, d'un ONT Nokia et d'un modem TP-Link souffrent de la même panne logique : un VLAN WAN mal configuré. L'agent formule dans les trois cas la même *intention* : « remettre le VLAN à la valeur correcte ». C'est la couche de résolution qui traduit cette intention en trois écritures physiquement différentes, comme le résume le tableau 4.5.

Table 4.5. Un même rôle logique résolu en trois chemins TR-069 distincts.

Modèle	Requête (rôle logique)	Écriture résolue
Huawei_8145	<code>set_vlan(100)</code>	suffixe <code>X_HW_VLAN</code>
NOKIA_1425	<code>set_vlan(100)</code>	suffixe <code>VLANIDMark</code>
TP-LINK_VC220-G3v	<code>set_vlan(100)</code>	suffixe <code>X_TP_VID</code>

Exemple

Lecture de l'exemple. Sans le graphe, il faudrait coder « en dur » une correspondance constructeur → chemin, à maintenir manuellement à chaque nouveau modèle ou mise à jour de firmware. En externalisant cette connaissance dans le graphe, l'ajout d'un modèle se réduit à l'ajout de nœuds et de relations : le code du moteur, lui, ne change pas. C'est précisément l'objectif « robustesse aux mises à jour firmware » du cahier des charges.

4.13.2 Exemple : calcul du statut dérivé

Le statut d'un routeur n'est jamais écrit directement : il est *recalculé* à partir des paramètres. Considérons l'ONT Huawei de l'exemple précédent, avant et après correction.

Table 4.6. Statut dérivé avant et après application du correctif.

Paramètre lu	Avant	Après
VLAN (<code>X_HW_VLAN</code>)	999	100
État PPP	Unconfigured	Connected
Diagnostic	<code>wrong_vlan</code>	aucun
Statut dérivé	DOWN	UP

Parce que le statut découle du diagnostic, un correctif qui n'éliminerait pas la cause laisserait le diagnostic actif et donc le statut à **DOWN** : l'escalade serait alors déclenchée. Aucune action ne peut « réussir faussement ».

4.13.3 Exemple : cheminement d'une requête dans l'architecture

Pour relier les vues d'architecture au fonctionnement réel, suivons une requête de bout en bout à travers les couches. (1) L'abonné envoie sa plainte au **portail client** (couche présentation). (2) Le **moteur de décision** identifie le client et son routeur, puis interroge **GenieACS** pour lire l'état (couche action, lecture). (3) Le moteur résout les rôles en chemins et diagnostique via **neo4j-agent** (couche connaissance). (4) Il déclenche **RaDuce** pour écrire le correctif (couche action, écriture), qui met à jour l'état dans **MongoDB** (couche données). (5) Le moteur relit l'état, confirme la résolution et notifie l'abonné ; le **tableau NOC** reflète l'ensemble en temps réel. Chaque couche n'a de responsabilité que sur son périmètre, ce qui garantit le faible couplage recherché.

4.14 Modélisation détaillée du graphe de connaissances

Le graphe étant la pièce maîtresse de la solution, il mérite une description approfondie de sa structure et des raisons qui l'ont façonnée.

Les nœuds. Le graphe distingue plusieurs types de nœuds. Les nœuds **Router** (ou **Model**) représentent les modèles d'équipements pris en charge — douze dans le périmètre du projet, couvrant plusieurs constructeurs. Les nœuds **Parameter** représentent les paramètres techniques individuels, chacun caractérisé par son chemin TR-069 réel, son type et un indicateur précisant s'il est modifiable ou en lecture seule. D'autres nœuds peuvent décrire des concepts métier (types de pannes, correctifs), reliant ainsi la connaissance descriptive à la connaissance procédurale.

Les relations. La richesse du graphe tient à ses relations typées. Une relation relie un modèle à chacun de ses paramètres ; c'est sur cette relation que se porte l'information la plus précieuse du projet : le *rôle canonique*. Ce rôle — par exemple `vlan_id`, `dns_servers` ou `ppp_connection_status` — constitue le vocabulaire logique et stable manipulé par l'agent, indépendamment des noms propriétaires que les constructeurs donnent aux paramètres. C'est cette relation porteuse de rôle qui rend possible la résolution rôle → chemin, et donc l'action multi-constructeurs.

Pourquoi porter le rôle sur la relation. On aurait pu attacher le rôle au nœud paramètre. Le porter sur la *relation* entre le modèle et le paramètre est un choix délibéré : un même chemin physique pourrait, en théorie, jouer des rôles légèrement différents selon les contextes, et surtout cette modélisation exprime clairement que « pour ce modèle, ce

paramètre joue *ce* rôle ». La connaissance devient ainsi contextuelle et sans ambiguïté, ce qui est exactement ce dont l'agent a besoin pour agir juste.

Le parcours à l'exécution. Lorsqu'à l'exécution l'agent a besoin d'agir sur un rôle, il n'effectue pas une recherche floue : il parcourt, depuis le nœud du modèle concerné, la relation portant le rôle recherché, et récupère le nœud paramètre cible avec son chemin exact et son caractère éditable. Cette opération, exprimée en une requête Cypher concise, est à la fois rapide et déterministe. Elle transforme une intention logique en une cible technique précise, prête à être lue ou écrite.

Enrichissement et évolutivité. L'un des principaux mérites de cette modélisation est sa capacité d'évolution. Prendre en charge un nouveau modèle de routeur ne demande pas de modifier le code du moteur : il suffit d'ajouter au graphe le nœud du modèle, les nœuds de ses paramètres et les relations porteuses de rôles correspondantes. De même, l'adaptation à une mise à jour de firmware qui déplacerait un paramètre se résout par une modification de données. Le code, lui, demeure stable. La connaissance vit dans le graphe ; le moteur ne fait qu'y naviguer.

Le graphe comme mémoire partagée. Enfin, le graphe joue un rôle de mémoire commune à l'ensemble des services : le moteur de décision, la couche d'action et l'interface de supervision s'appuient sur la même source de vérité. Cette unicité évite les divergences et garantit que ce que l'agent croit vrai, ce qu'il écrit et ce que l'ingénieur observe reposent sur une seule et même représentation de la connaissance.

4.15 Conclusion

Ce chapitre a présenté la conception de la plateforme, des architectures aux modèles de données, en insistant sur la résolution rôle → chemin qui rend le graphe déterminant. Le chapitre suivant décrit la réalisation concrète de ces choix.

CHAPITRE 5

Réalisation

5.1 Introduction

Ce chapitre décrit la réalisation de la plateforme : l'environnement de travail, la pile technologique, le déploiement conteneurisé, l'implémentation des composants clés, les interfaces développées (portail client et tableau de bord NOC), un scénario de démonstration de bout en bout, ainsi que la validation du système.

5.2 Environnement de travail

5.2.1 Environnement matériel

Le développement et les essais ont été menés sur un poste de travail standard, l'exécution locale du LLM (Qwen 2.5 via Ollama) constituant le principal facteur dimensionnant en mémoire et en calcul.

5.2.2 Environnement logiciel et pile technologique

La solution s'appuie sur une pile technologique moderne (figure 5.1) : Python et le framework Flask/FastAPI pour les services, Neo4j pour le graphe de connaissances, MongoDB pour la persistance, Ollama et Qwen 2.5 pour l'inférence LLM locale, GenieACS pour la télégestion TR-069, le Model Context Protocol pour l'orchestration d'outils, et Docker / Docker Compose pour la conteneurisation.

5.3 Déploiement conteneurisé

L'ensemble des services est orchestré par Docker Compose et supervisé via Docker Desktop (figure 5.2). Chaque service dispose de son `Dockerfile` et communique sur le réseau `mypfe-net` ; la base Neo4j s'exécute sur l'hôte et est jointe via `host.docker.internal`.

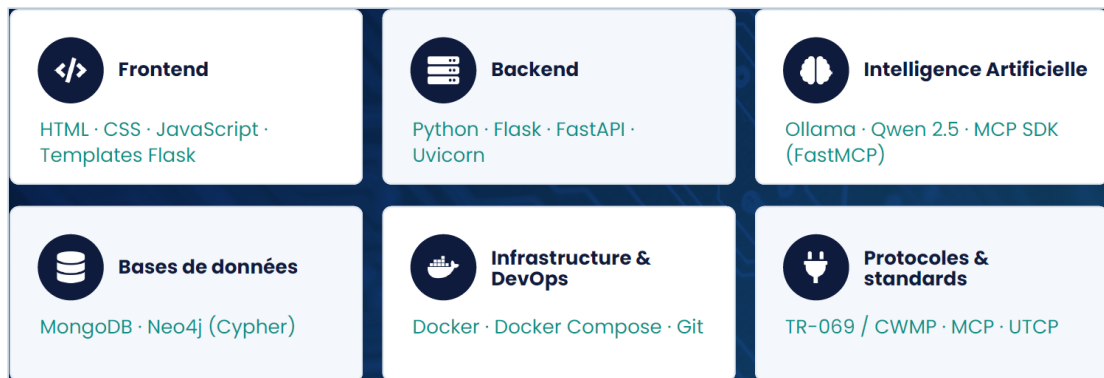


Figure 5.1. Outils et technologies mobilisés.

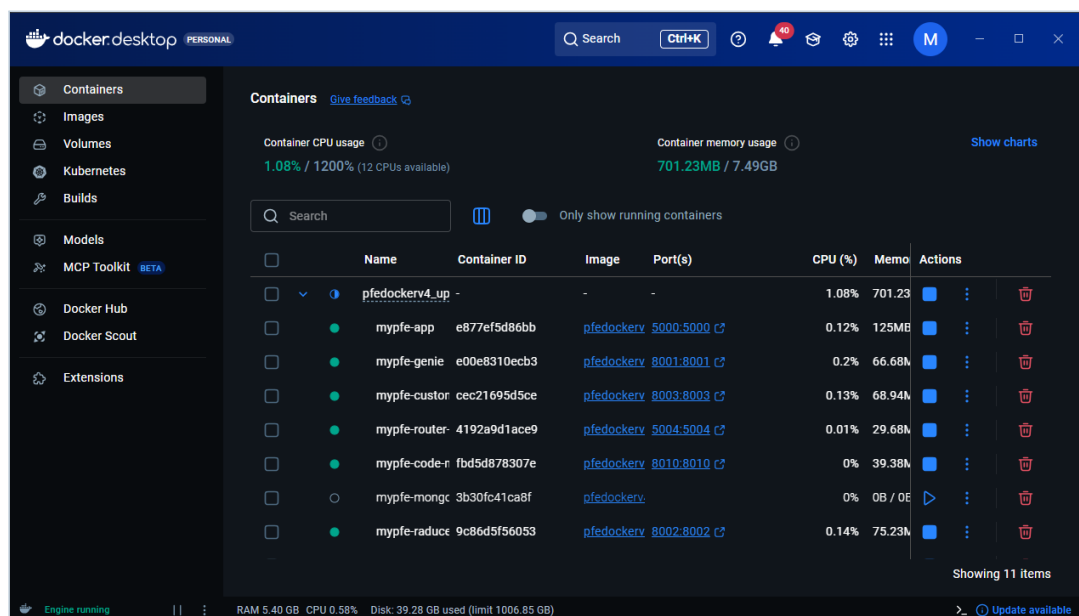


Figure 5.2. Supervision des conteneurs sous Docker Desktop.

5.4 Implémentation des composants clés

5.4.1 Moteur de décision et couche GraphRAG

Le moteur de décision (service `mypfe-app`) orchestre la boucle fermée. Le diagnostic est **piloté par le graphe** : pour chaque rôle de lecture, la valeur est récupérée à son chemin TR-069 puis confrontée à un ensemble de règles déterministes. Le diagnostic est **déterministe** et suit une cascade de règles évaluées dans un ordre de priorité. Pour chaque rôle de lecture, la valeur est récupérée à son chemin TR-069 et confrontée aux conditions suivantes, la première satisfaite l'emportant :

1. si la puissance optique reçue est inférieure au seuil critique (-27 dBm), la panne est physique : *escalade* ;
2. si le compteur d'erreurs CRC dépasse le seuil, la panne est matérielle : *escalade* ;
3. si le VLAN lu diffère de la valeur attendue, la panne est `wrong_vlan` : correctif `set_vlan` ;
4. si les serveurs DNS sont invalides, la panne est `dns_failure` : correctif `set_dns` ;
5. si l'état PPP est déconnecté, la panne relève de l'authentification ou d'un aléa : correctif `restart_ppp` ;
6. sinon, le routeur est déclaré sain.

À chaque étape, le rôle lu, son chemin TR-069 et la valeur relevée sont ajoutés à une *trace* — la « trace GraphRAG » — restituée au NOC pour l'audit.

5.4.2 GenieACS : la lecture de l'état

Le service `genie` joue le rôle du serveur ACS. Il expose, via une interface REST/MCP, l'état courant d'un routeur sous forme de paramètres indexés par leurs chemins TR-069 réels, accompagnés d'une vue lisible (rôle $\rightarrow$ valeur) et du statut dérivé.

5.4.3 RaDuce : le moteur de remédiation

Le service `raduce` exécute les correctifs. Chaque correctif résout d'abord le rôle en chemin TR-069 via le graphe, vérifie que le paramètre est éditable (liste blanche), écrit la nouvelle valeur au chemin, puis recalcule le statut. Cette conception garantit qu'un même correctif logique s'applique au bon chemin selon le constructeur, et qu'une action sur un paramètre

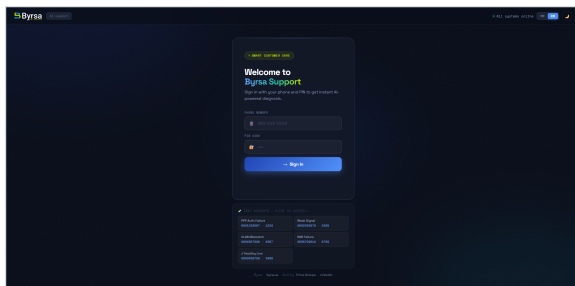
en lecture seule est refusée. Il convient d'insister sur la portée de ce dernier point. La liste blanche n'est pas un simple filtre de confort : c'est une *barrière de sécurité*. Un agent fondé sur un LLM demeure, par nature, non déterministe ; il pourrait, dans un cas limite, proposer une action inappropriée. En n'autorisant l'écriture que sur un ensemble restreint et explicitement déclaré de rôles éditables, le système borne *a priori* l'espace des actions possibles. Même une décision erronée du modèle ne peut donc franchir cette barrière. On obtient ainsi le meilleur des deux mondes : la souplesse de raisonnement d'un LLM, encadrée par la rigueur déterministe d'une couche d'action vérifiée.

5.4.4 Boucle fermée et sûreté

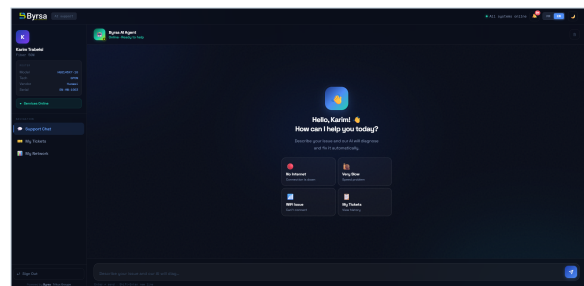
L'enchaînement *lire* $\rightarrow$ *diagnostiquer* $\rightarrow$ *corriger* $\rightarrow$ *vérifier* constitue la boucle fermée. Le statut étant dérivé, la vérification relit réellement l'état : si la panne persiste, l'escalade est déclenchée. Les pannes non réparables à distance (signal optique faible, erreurs de ligne matérielles) sont directement orientées vers l'escalade. Cette conception mérite d'être soulignée car elle inverse une logique habituelle. Dans beaucoup de systèmes automatisés, l'action est considérée comme réussie dès qu'elle a été *émise* ; ici, elle n'est réputée réussie que lorsque son *effet* a été constaté. Le système ne se contente donc pas d'« essayer » : il vérifie, et n'affirme la résolution que si l'état observé le confirme. Cette exigence de preuve est ce qui autorise à laisser l'agent agir en autonomie sans compromettre la fiabilité.

5.5 Interfaces réalisées : portail client (front office)

Le portail client offre une expérience conversationnelle simple et sécurisée. L'abonné s'authentifie par téléphone et code PIN (figure 5.3a), avec vérification par e-mail (figures 5.5a et 5.5b). La page principale (figure 5.3b) et la page promotionnelle (figure 5.4) présentent les services offerts.



(a) Page d'authentification.



(b) Page principale du portail.

Figure 5.3. Portail client : authentification et page d'accueil.

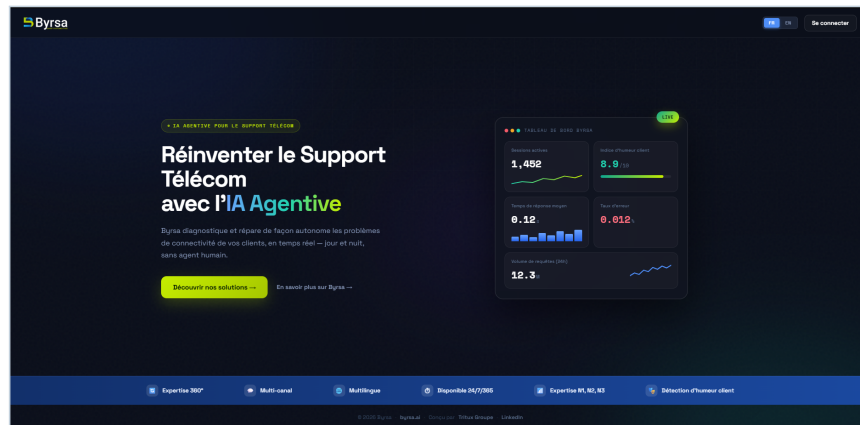
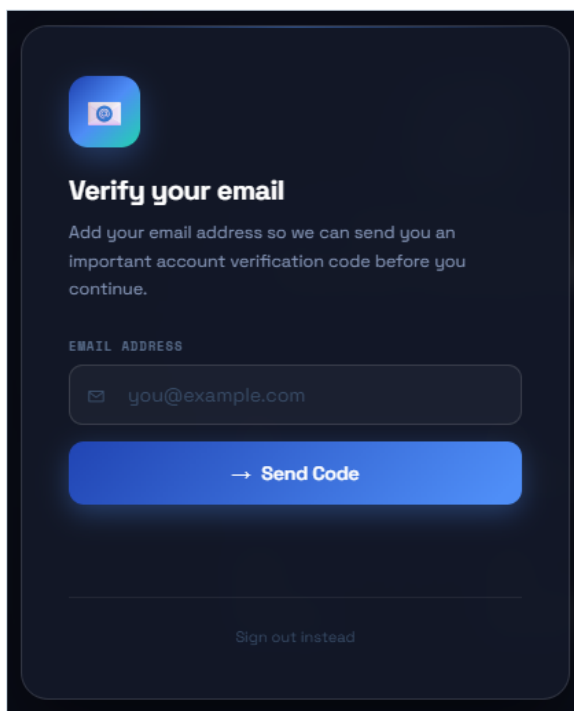
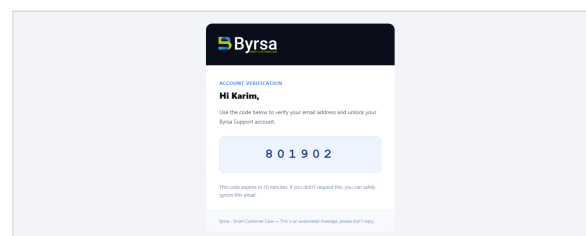


Figure 5.4. Page promotionnelle du portail client.



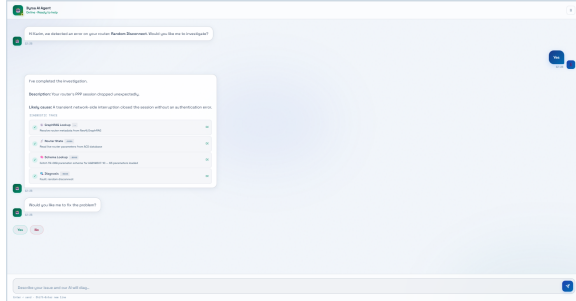
(a) Vérification par e-mail.



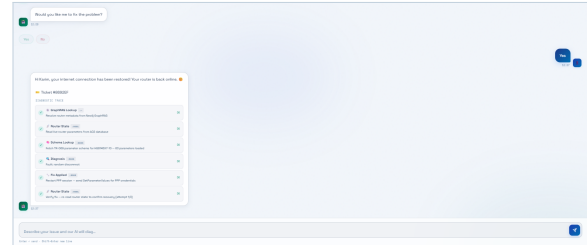
(b) Courriel de vérification reçu.

Figure 5.5. Processus de vérification de l'abonné par e-mail.

Le cœur de l'expérience est l'assistant conversationnel : l'abonné décrit sa panne en langage naturel et l'agent lui répond en menant le diagnostic et la correction (figures 5.6a et 5.6b). L'abonné suit ensuite ses tickets (figure 5.7a), reçoit des notifications (figure 5.7b) et consulte des statistiques (figure 5.7c).

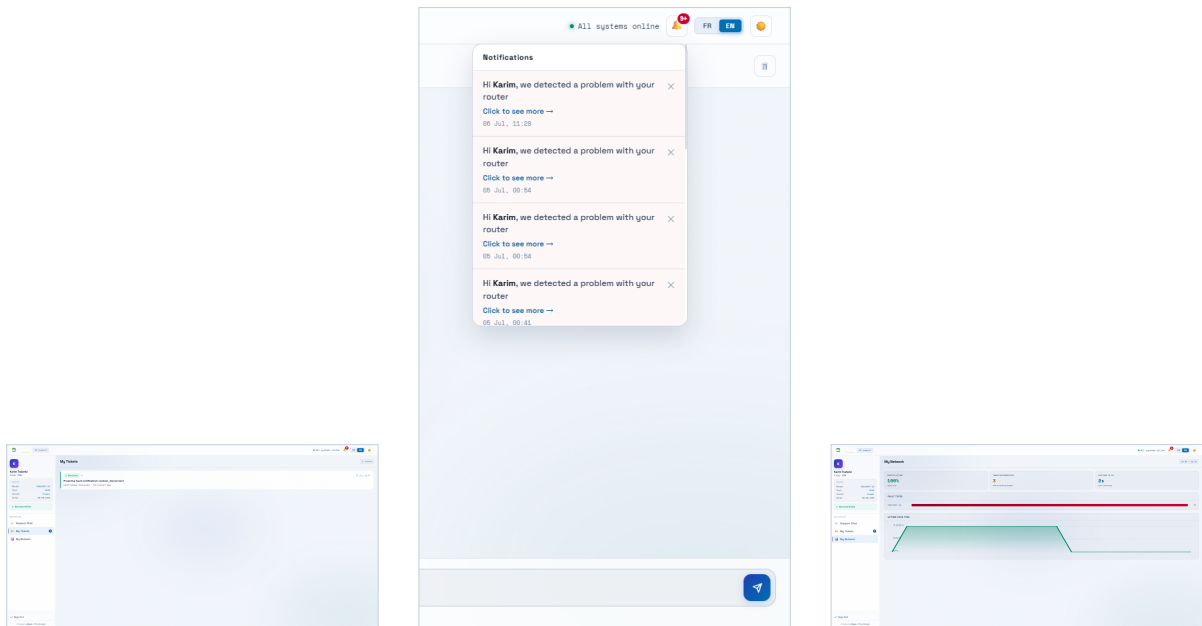


(a) Dialogue avec l'assistant (1).



(b) Dialogue avec l'assistant (2).

Figure 5.6. Assistant conversationnel : diagnostic et correction en langage naturel.



(a) Liste des tickets.

(b) Notifications.

(c) Statistiques abonné.

Figure 5.7. Suivi des tickets, notifications et statistiques côté abonné.

Enfin, le système informe proactivement l'abonné par courriel en cas de problème détecté sur son routeur (figure 5.8).

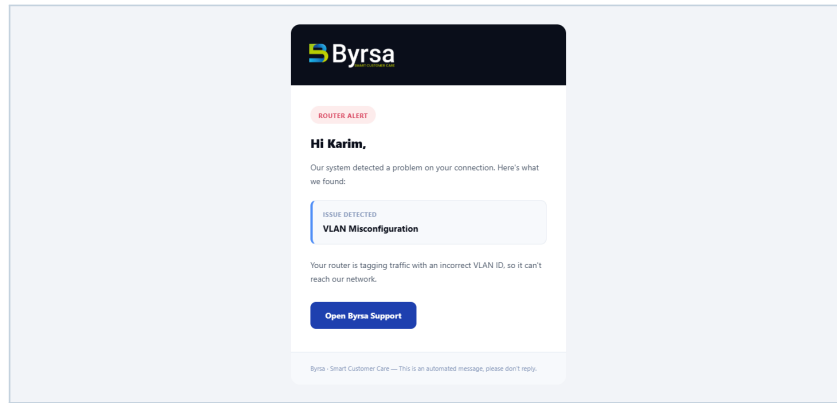
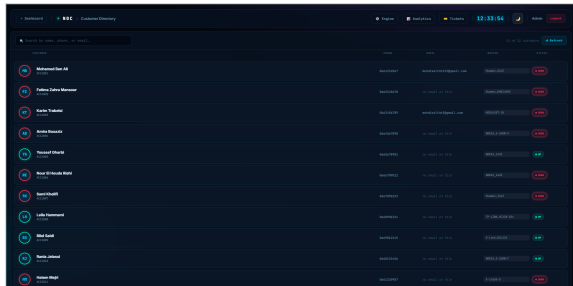


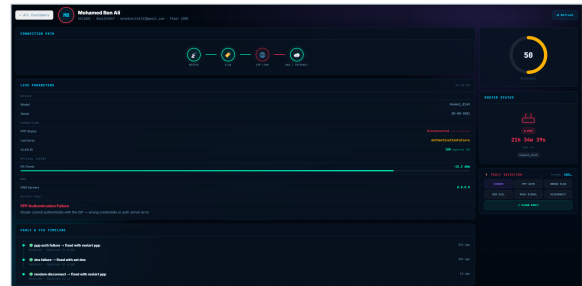
Figure 5.8. Notification par courriel d'un problème détecté sur le routeur.

5.6 Interfaces réalisées : tableau de bord NOC (back office)

Le tableau de bord NOC offre aux ingénieurs une vue complète du parc. Il permet de lister les abonnés (figure 5.9a) et d'en consulter le détail (figure 5.9b), de gérer et rechercher les tickets (figure 5.10a), et d'accéder à l'interface de supervision (figure 5.10b).

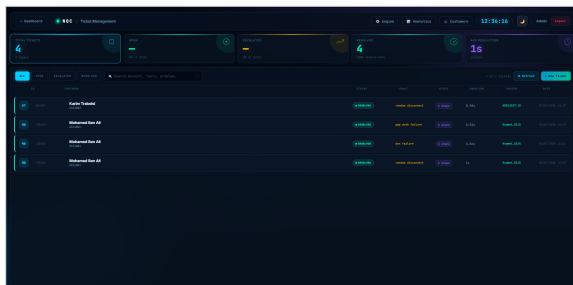


(a) Liste des abonnés.

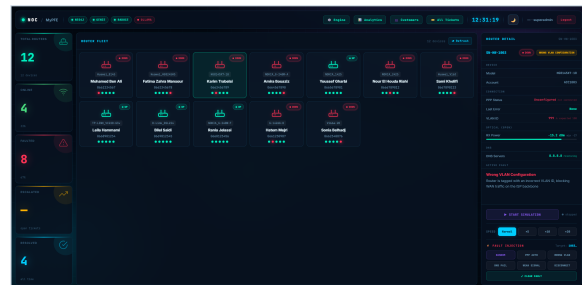


(b) Détail d'un abonné.

Figure 5.9. Gestion des abonnés côté NOC.



(a) Gestion des tickets.



(b) Interface de supervision NOC.

Figure 5.10. Supervision et gestion des incidents côté NOC.

Un apport important pour la transparence est la **visualisation du raisonnement de l'IA** : le NOC peut suivre, étape par étape, le diagnostic mené (figure 5.11a), y compris

les chemins TR-069 réellement lus. Le NOC dispose également d'un **simulateur d'état et d'un injecteur de pannes** pour tester le système (figure 5.11b) et de statistiques par routeur (figure 5.12).

5.7 Scénario de démonstration de bout en bout

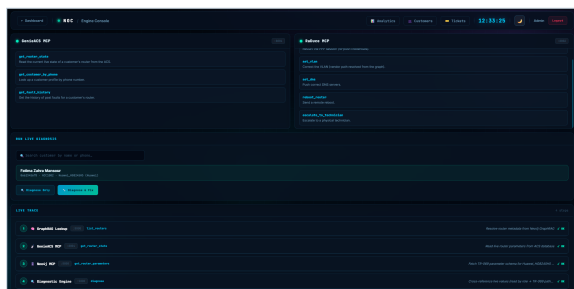
La figure 5.13 illustre un scénario complet de résolution autonome. (1) Le client saisit « Mon Wi-Fi est coupé ». (2) L'IA identifie le client, interroge GenieACS et détecte l'anomalie. (3) Via GraphRAG, elle retrouve le chemin de configuration exact pour ce modèle. (4) Elle déclenche RaDuce via MCP pour rétablir le paramètre correct. (5) Le système confirme la résolution au client et clôture le ticket.

5.8 Tests et validation

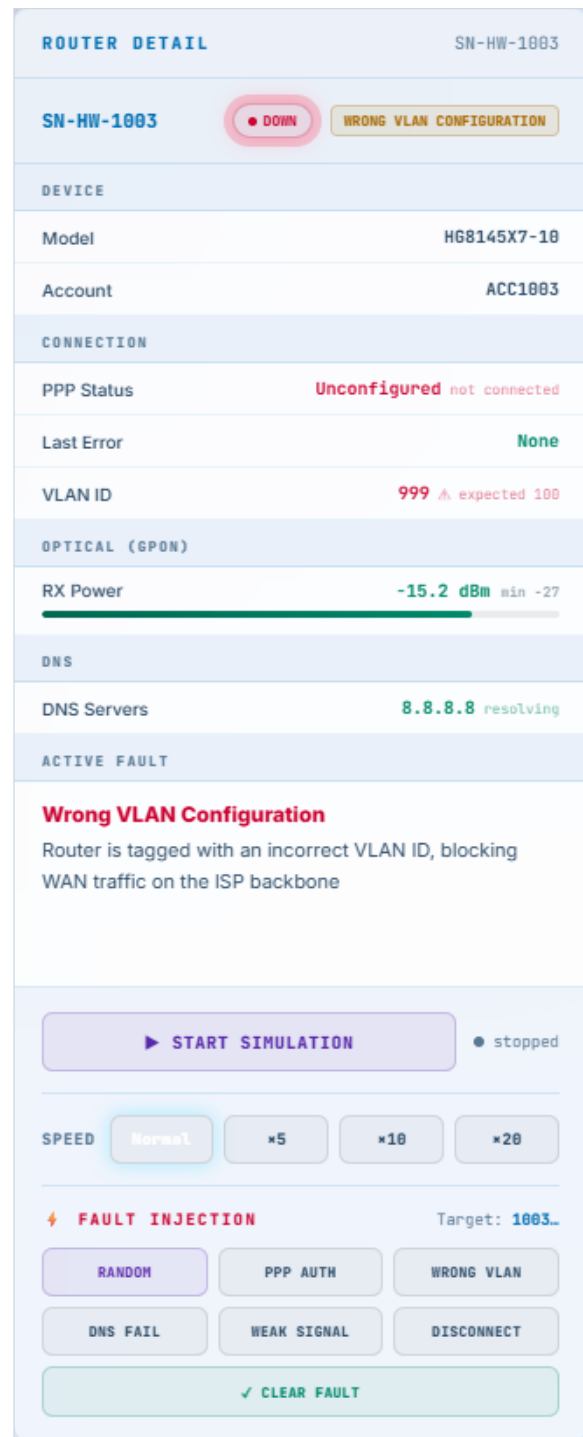
La plateforme a été validée sur un **banc d'essai reproduisant douze modèles de routeurs réels** (Huawei, Nokia, TP-Link, D-Link), couvrant les technologies GPON et xDSL. Pour chaque type de panne injecté, le système a été observé sur la boucle complète *injection* → *diagnostic* → *correction* → *vérification*. Les pannes réparables à distance (échec d'authentification PPP, VLAN mal configuré, panne DNS) ont été résolues automatiquement en écrivant le bon chemin propre à chaque constructeur ; les pannes physiques (signal optique faible, erreurs de ligne) ont été correctement escaladées. Le mécanisme de liste blanche a par ailleurs bien refusé toute tentative d'écriture sur un paramètre en lecture seule. Ces essais ont également permis de vérifier une propriété plus subtile mais essentielle : la *répétabilité*. Rejouer le même scénario sur des modèles de constructeurs différents produit systématiquement la bonne écriture au bon chemin, sans adaptation de code, ce qui valide empiriquement l'apport du graphe. De même, un correctif volontairement rendu inefficace laisse le statut à **DOWN** et déclenche l'escalade, confirmant que la boucle de vérification joue bien son rôle de garde-fou. Au-delà de la simple réussite fonctionnelle, ce sont ces propriétés — répétabilité, sûreté, honnêteté du résultat — qui donnent confiance dans le système.

5.9 Procédure de déploiement

Le déploiement de la plateforme suit une procédure reproductible. Les rôles sont d'abord estampillés dans le graphe Neo4j (script `migrate_roles.py`) et les chemins TR-069 standards manquants ajoutés. L'ensemble des microservices est ensuite construit et démarré



(a) Raisonnement de l'IA (diagnostic).



(b) Simulateur et injecteur de pannes.

Figure 5.11. Observabilité et tests : raisonnement de l'IA et injection de pannes.



Figure 5.12. Statistiques détaillées par routeur.

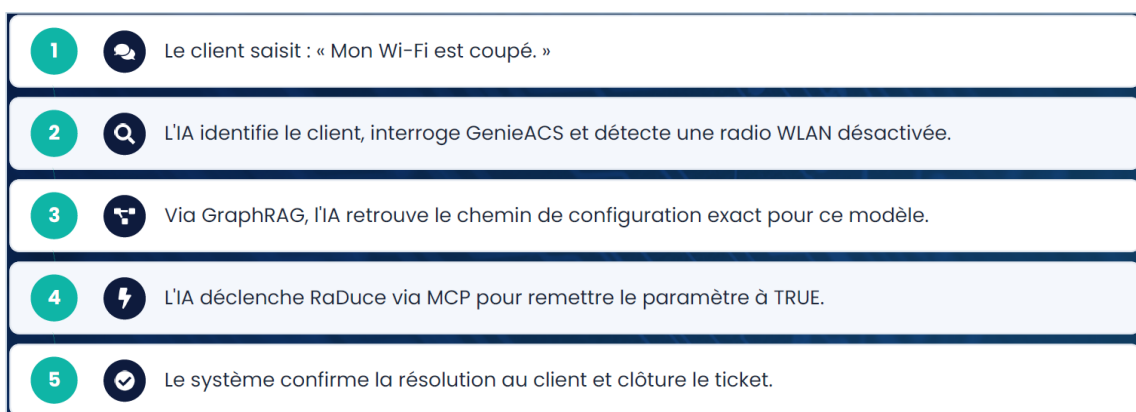


Figure 5.13. Scénario de bout en bout : de la plainte à la résolution autonome.

par une unique commande Docker Compose, puis la base MongoDB est initialisée avec l'état des douze routeurs (`seed.py`). Enfin, la résolution rôle $\rightarrow$ chemin est vérifiée en interrogeant le point d'accès dédié de l'agent Neo4j. En cas d'indisponibilité du graphe, le système bascule automatiquement sur une carte de chemins mise en cache, assurant la continuité de service. Ce mécanisme de repli mérite d'être souligné, car il traduit un souci de robustesse opérationnelle. Un système qui s'effondrerait à la moindre indisponibilité d'un de ses composants serait inutilisable en production. En prévoyant un mode dégradé — où la connaissance récemment résolue reste exploitable même sans accès au graphe — on garantit que le support continue de fonctionner, quitte à perdre temporairement la capacité d'apprendre de nouveaux chemins. Cette dégradation *gracieuse* est une caractéristique attendue des systèmes industriels fiables.

5.10 Plan de tests et validation

La validation a couvert la boucle complète pour chaque type de panne. Le tableau 5.1 synthétise les principaux cas de test et leurs résultats attendus.

Table 5.1. Plan de tests fonctionnels de la boucle de remédiation.

#	Cas de test	Résultat attendu	Statut
T1	VLAN incorrect (Huawei)	Écriture sur <code>X_HW_VLAN</code> , retour à l'état UP	✓
T2	VLAN incorrect (TP-Link)	Écriture sur <code>X_TP_VID</code> , retour à l'état UP	✓
T3	VLAN incorrect (Nokia)	Écriture sur <code>VLANIDMark</code> , retour à l'état UP	✓
T4	Échec authentification PPP	<code>restart_ppp</code> , reconnexion, état UP	✓
T5	Panne DNS	<code>set_dns</code> , résolution, état UP	✓
T6	Signal optique faible	Escalade technicien, ticket ouvert	✓
T7	Erreurs de ligne (xDSL)	Escalade technicien, ticket ouvert	✓
T8	Écriture sur paramètre en lecture seule	Refus par la liste blanche	✓
T9	Correctif inefficace	Statut reste DOWN, escalade déclenchée	✓

5.11 Illustration d'une résolution tracée

Pour illustrer l'observabilité offerte au NOC, considérons la résolution d'un VLAN mal configuré sur un routeur Huawei. La trace produite par le diagnostic — la « trace GraphRAG » — expose les rôles lus, les chemins TR-069 correspondants et les valeurs relevées, suivis de l'action corrective :

```

1 [lire]      get_router_state(SN-HW-1001)
2 [diagnostic] role=ppp_connection_status path=...ConnectionStatus value="
    ↳ Unconfigured"
3           role=vlan_id path=...X_HW_VLAN value=999
4           => panne detectee : wrong_vlan
5 [correctif] set_vlan(SN-HW-1001, 100)
6           ecriture path=...X_HW_VLAN <- 100
7 [verif]     statut derive = UP (aucune panne restante)
8 [resultat]  ticket clos, abonne notifie

```

Listing 5.1. Trace d'une résolution (VLAN incorrect, Huawei_8145).

Cet exemple met en évidence le rôle déterminant du graphe : c'est lui qui a fourni le chemin `X_HW_VLAN`, spécifique à Huawei, sans lequel l'écriture aurait été impossible.

5.12 Justification des choix technologiques

Les technologies retenues ne l'ont pas été par mode mais par adéquation au problème. Cette section explicite les raisons de chaque grand choix.

Un LLM local plutôt qu'en nuage. Le support télécom manipule des données personnelles d'abonnés ; les confier à un service tiers poserait un problème de confidentialité et de souveraineté, sans compter un coût par appel qui croît avec le volume. L'exécution locale, via Ollama et un modèle de taille modérée comme Qwen 2.5, écarte ces écueils. La moindre puissance brute est compensée par la qualité du contexte fourni par le graphe : on ne demande pas au modèle de tout savoir, mais de bien raisonner sur des faits exacts.

Neo4j plutôt qu'une base relationnelle. Le problème est intrinsèquement relationnel : naviguer entre modèles, paramètres, rôles et dépendances. Un modèle graphe épouse cette structure et rend les requêtes de parcours naturelles et performantes, là où une base relationnelle multiplierait les jointures. Le langage Cypher exprime en outre ces parcours de façon lisible, ce qui facilite la maintenance de la connaissance.

MongoDB pour l'état des équipements. L'état d'un routeur est un document semi-structuré, indexé par des chemins TR-069 hétérogènes d'un constructeur à l'autre. Une base documentaire s'y prête mieux qu'un schéma relationnel rigide : elle accueille sans friction la diversité des paramètres et évolue avec le parc.

Docker et les microservices. La solution assemble des briques de natures très différentes — inférence, graphe, base documentaire, serveurs d'outils, interface. Les conteneuriser et les isoler en microservices permet de les développer, de les déployer et de les faire monter en charge indépendamment, tout en garantissant un environnement d'exécution reproductible, identique en local et en production.

MCP et « code mode » pour l'orchestration. Exposer les actions via un protocole standardisé confère à l'agent une interface uniforme et une découverte dynamique des outils, réduisant le couplage. Là où plusieurs actions s'enchaînent, l'approche « code mode » les regroupe en une routine unique, réduisant la latence et la consommation de tokens — un atout pour un modèle local.

TR-069 comme socle d'action. S'appuyer sur un standard de télégestion éprouvé, plutôt que sur des interfaces propriétaires, garantit l'interopérabilité avec un large éventail d'équipements et inscrit la solution dans un écosystème pérenne. La variabilité résiduelle entre constructeurs est précisément ce que le graphe absorbe.

Cohérence d'ensemble. Aucun de ces choix n'est isolé : ils se renforcent mutuellement autour d'un même principe directeur — externaliser la connaissance, borner l'action, vérifier les effets. C'est cette cohérence, plus que la modernité de chaque brique prise séparément, qui fait la solidité de l'architecture proposée.

5.13 Scénario de bout en bout commenté

Pour rassembler en un seul récit l'ensemble des mécanismes décrits dans ce chapitre, déroulons intégralement le traitement d'un incident, depuis la plainte de l'abonné jusqu'à la clôture du ticket. Ce scénario, volontairement détaillé, met en évidence la coopération des services et la circulation de l'information à travers les couches.

Point de départ. Un abonné équipé d'un ONT Nokia constate qu'il n'a plus accès à Internet. Il ouvre le portail, s'authentifie à l'aide de son numéro de téléphone et de son

code PIN, puis confirme son identité via le code reçu par courriel. Une session sécurisée s'ouvre. Il écrit simplement à l'assistant : « je n'ai plus internet depuis tout à l'heure ».

Compréhension de la demande. L'assistant, porté par le LLM local, interprète l'intention. Il ne s'arrête pas aux mots employés : il rattache la formulation familière à la catégorie technique « perte de connectivité WAN ». Il identifie l'abonné à partir de sa session, retrouve le numéro de série de son équipement et le modèle associé. À ce stade, aucune action n'a encore été entreprise sur le réseau ; le système sait seulement *qui* se plaint et *de quel type de problème* il s'agit.

Lecture de l'état réel. Le moteur de décision sollicite alors le service de lecture (GenieACS) pour relever l'état courant de l'équipement. Mais pour savoir *quels* paramètres lire, il doit d'abord résoudre les rôles pertinents en chemins TR-069. Il interroge le graphe : pour ce modèle Nokia, le rôle `vlan_id` correspond au chemin se terminant par `VLANIDMark`, et le rôle `ppp_connection_status` à son propre chemin. Le moteur lit ces valeurs et découvre un VLAN incohérent avec la valeur attendue, tandis que l'état PPP est inactif.

Diagnostic. Le moteur applique alors sa cascade de règles déterministes. La puissance optique reçue est normale : ce n'est pas une panne physique. Les erreurs de ligne sont sous le seuil : ce n'est pas une panne matérielle. En revanche, le VLAN lu diffère de la valeur attendue : le diagnostic conclut à une panne `wrong_vlan`. À chaque étape, le rôle consulté, son chemin et la valeur relevée sont consignés dans la trace de raisonnement — la « trace GraphRAG » — qui sera visible par le NOC.

Remédiation. Le diagnostic `wrong_vlan` appelle le correctif `set_vlan`. Le moteur formule cette intention et la transmet à la couche d'action d'écriture (RaDuce). Celle-ci vérifie d'abord que le rôle `vlan_id` figure bien dans la liste blanche des paramètres éditables — ce qui est le cas — puis résout à nouveau le chemin propre au constructeur et écrit la valeur correcte à l'emplacement `VLANIDMark`. L'écriture est confirmée par le protocole.

Vérification. L'action n'est pas pour autant réputée réussie. Le moteur relit l'état de l'équipement : le VLAN affiche désormais la bonne valeur, l'état PPP est repassé à actif. Il recalcule alors le statut dérivé : aucun diagnostic ne détectant plus de panne, le routeur est déclaré UP. La résolution est *prouvée*, non simplement supposée.

Clôture et notification. Le système met à jour l'état persisté, crée un ticket tracé pour l'historique, informe l'abonné que sa connexion est rétablie et lui adresse un courriel récapitulatif. L'abonné, qui n'a rien perçu de la complexité technique sous-jacente, constate simplement que tout fonctionne à nouveau.

Vue côté opérateur. Pendant ce temps, l'ingénieur NOC a pu suivre l'incident sur son tableau de bord : apparition de l'équipement en panne, déroulé du raisonnement avec les chemins consultés, application du correctif, retour au statut opérationnel. S'il l'avait souhaité, il aurait pu intervenir manuellement ; le système lui a laissé ce contrôle tout en faisant l'essentiel du travail.

Cas d'une panne non réparable. Imaginons maintenant une variante : la puissance optique reçue est descendue sous le seuil critique. Dès la phase de diagnostic, le moteur identifie une panne *physique* qu'aucune écriture ne saurait corriger. Il n'entreprend alors aucune action logicielle : il ouvre un ticket, escalade vers un technicien et en informe l'abonné avec transparence. Ce contraste illustre la frontière, tracée dès la conception, entre ce que le système répare et ce qu'il confie à l'humain — frontière qui fonde la confiance qu'on peut lui accorder.

5.14 Exemples d'interactions et de résolutions

Cette section illustre le comportement du système par des exemples concrets, du dialogue avec l'abonné jusqu'au traitement de chaque famille de pannes.

5.14.1 Exemple : un dialogue complet avec l'assistant

Le tableau 5.2 reproduit un échange typique entre un abonné et l'assistant lors d'une coupure due à un VLAN mal configuré. Le ton reste simple et rassurant côté abonné, tandis que le travail technique se déroule en arrière-plan.

5.14.2 Exemples par famille de pannes

Le système traite six familles de pannes, réparties entre correction automatique à distance et escalade. Les paragraphes suivants détaillent, pour chacune, les valeurs observées et l'action entreprise.

Table 5.2. Exemple de dialogue abonné ↔ assistant (coupure VLAN).

Interlocuteur	Message
Abonné	« Bonjour, je n'ai plus internet depuis ce matin. »
Assistant	« Bonjour. Je vérifie votre équipement, un instant... »
(système)	<i>lecture de l'état via GenieACS ; diagnostic : <code>wrong_vlan</code>.</i>
Assistant	« J'ai détecté un problème de configuration réseau sur votre box. Je le corrige à distance. »
(système)	<i>résolution <code>vlan_id</code> → chemin, écriture via RaDuce, vérification.</i>
Assistant	« C'est réparé : votre connexion est de nouveau active. Un ticket a été créé pour la traçabilité. »
Abonné	« Super, merci ! »

Échec d'authentification PPP. L'état PPP est `Disconnected` avec une dernière erreur de type *authentication*. Le diagnostic conclut à `ppp_auth_failure` ; le correctif `restart_ppp` re-pousse les identifiants et relance la session. Après vérification, l'état repasse à `Connected`.

Déconnexions aléatoires. L'état PPP est `Disconnected` sans erreur d'authentification (délai serveur). Le diagnostic `random_disconnect` déclenche également `restart_ppp`, qui rétablit la session.

VLAN mal configuré. La valeur du VLAN lue (par exemple 999) diffère de la valeur attendue. Le correctif `set_vlan` écrit la bonne valeur au chemin propre au constructeur, comme détaillé au chapitre 4.

Panne DNS. Les serveurs DNS valent 0.0.0.0. Le correctif `set_dns` pousse des résolveurs valides ; la navigation redevient fonctionnelle.

Signal optique faible. La puissance optique reçue passe sous -27 dBm. Aucune action logicielle ne peut compenser une atténuation physique : le système ouvre un ticket et **escalade** vers un technicien.

Erreurs de ligne matérielles. Le compteur d'erreurs CRC dépasse le seuil. Le diagnostic `hardware_fault` conduit lui aussi à une escalade, car la cause est matérielle.

Exemple

Frontière automatique / humain. La force du système ne réside pas seulement dans ce qu'il répare, mais dans ce qu'il *refuse* de prétendre réparer. En orientant sans détour les pannes physiques vers un technicien, il évite les fausses résolutions et préserve la confiance de l'abonné — un critère essentiel d'acceptabilité pour un opérateur.

5.14.3 Exemple : l'observabilité offerte au NOC

Lorsqu'un ingénieur NOC ouvre la vue de raisonnement (figure 5.11a), il ne voit pas une simple réponse « réparé / non réparé » : il suit la chaîne complète — rôles lus, chemins TR-069 correspondants, valeurs relevées, panne déduite, correctif choisi, statut après vérification. Cette transparence transforme l'IA d'une « boîte noire » en un **assistant auditable** : l'ingénieur peut valider le raisonnement, détecter un cas limite et, au besoin, intervenir manuellement. C'est aussi un puissant outil de formation pour les agents de niveau 1. En donnant à voir, pas à pas, comment un incident est diagnostiqué et résolu, cette vue agit comme un mentor implicite : un agent débutant peut y observer la logique experte à l'œuvre et l'assimiler. Le système ne se contente donc pas de remplacer certaines tâches ; il contribue aussi à *élever* la compétence des équipes, ce qui en fait un outil d'accompagnement autant que d'automatisation.

5.15 Parcours détaillé des interfaces

Au-delà de leur présentation, les interfaces méritent une description de leur logique d'usage.

5.15.1 Portail abonné, écran par écran

L'**écran d'authentification** (figure 5.3a) demande le numéro de téléphone et le code PIN ; il constitue le point d'entrée sécurisé, complété par une vérification par e-mail qui confirme que l'abonné est bien le titulaire du compte. La **page principale** (figure 5.3b) offre un accès immédiat à l'assistant conversationnel, au suivi des tickets et aux informations de l'abonnement, dans une mise en page épurée pensée pour un public non technique. La **page promotionnelle** (figure 5.4) valorise les services de l'opérateur sans surcharger le parcours de support.

Le cœur de l'expérience est l'**assistant** (figures 5.6a et 5.6b). L'abonné y écrit librement ; l'agent répond en langage naturel tout en pilotant, de façon invisible, le diagnostic et la correction. La **liste des tickets** (figure 5.7a) donne à l'abonné une vision transparente de l'historique et de l'état de ses demandes, tandis que les **notifications** (figure 5.7b) l'informent proactivement. Les **statistiques** (figure 5.7c) restituent, de manière lisible, l'état et la qualité de sa connexion. Enfin, en cas de détection d'un problème, l'abonné reçoit un **courriel** explicatif (figure 5.8), ce qui referme la boucle de communication.

5.15.2 Tableau de bord NOC, écran par écran

Côté opérateur, la **liste des abonnés** (figure 5.9a) et la **fiche détaillée** (figure 5.9b) offrent une vue complète du parc et de chaque client. La **gestion des tickets** (figure 5.10a) permet de rechercher, filtrer et suivre les incidents. L'**interface de supervision** (figure 5.10b) présente l'état du parc en temps réel.

L'apport distinctif est la **vue de raisonnement** (figure 5.11a), qui expose la chaîne de diagnostic et les chemins TR-069 réellement consultés : l'ingénieur peut ainsi auditer chaque décision de l'IA. Le **simulateur et injecteur de pannes** (figure 5.11b) constitue un banc de test intégré : il permet de provoquer une panne contrôlée et d'observer la remédiation de bout en bout. Enfin, les **statistiques par routeur** (figure 5.12) fournissent une vision fine, équipement par équipement, utile au pilotage de la qualité de service.

5.15.3 Environnement technique de la réalisation

Le tableau 5.3 récapitule les principaux éléments de l'environnement de développement et d'exécution.

5.16 Conclusion

Ce chapitre a présenté la réalisation concrète de la plateforme, de l'environnement de déploiement aux interfaces abonné et NOC, jusqu'à la validation sur banc d'essai. Le système met en œuvre, de bout en bout, la boucle fermée d'automatisation conçue aux chapitres précédents.

Table 5.3. Environnement technique du projet.

Élément	Choix retenu
Langage principal	Python
Services web	Flask / FastAPI
Modèle de langage	Qwen 2.5 (via Ollama, local)
Graphe de connaissances	Neo4j (Cypher)
Base de données	MongoDB
Télégestion	GenieACS (TR-069), simulé sur le banc
Orchestration d'outils	Model Context Protocol (MCP), UTCP « code mode »
Conteneurisation	Docker, Docker Compose

CHAPITRE 6

Évaluation, discussion et perspectives

6.1 Introduction

Ce chapitre prend du recul sur le travail réalisé. Il confronte les résultats obtenus aux objectifs initiaux, discute honnêtement les limites de la solution, analyse ses propriétés de sécurité et de scalabilité, puis détaille les perspectives d'évolution.

6.2 Bilan des réalisations

Plusieurs résultats concrets ont été atteints. La documentation technique de douze modèles de routeurs a été modélisée en un graphe Neo4j exploitable. Une boucle agentique complète d'appel d'outils via le Model Context Protocol a été mise en œuvre, du raisonnement du LLM local jusqu'à l'action réelle sur l'équipement. Les pipelines de raisonnement sont localisés et à faible latence. Enfin, une interface double — portail abonné et tableau de bord NOC — a été développée et validée.

6.3 Confrontation aux objectifs

Le tableau 6.1 met en regard les objectifs annoncés au chapitre 1 et leur niveau de réalisation.

6.4 Discussion des limites

La lucidité sur les limites est essentielle à la valeur scientifique du travail. Trois points méritent d'être soulignés. Premièrement, le banc d'essai repose sur un serveur ACS **simulé** au-dessus de MongoDB : bien que fidèle à l'arborescence TR-069 et aux chemins réels par constructeur, il ne substitue pas encore un GenieACS pilotant des CPE physiques. Deuxièmement, les **gains quantitatifs** (taux de résolution automatique, MTTR moyen, taux d'escalade) restent à mesurer systématiquement sur un déploiement à plus grande échelle et sur une durée représentative. Troisièmement, le **périmètre des**

Table 6.1. Objectifs initiaux et niveau de réalisation.

Objectif	État	Commentaire
Moteur d'IA agentique (LLM local)	Réalisé	Ollama / Qwen 2.5, 100 % local.
Documentation en graphe Neo4j	Réalisé	12 modèles, relations porteuses de rôles.
Couche d'action MCP sur GenieACS	Réalisé	Lecture (Genie) et écriture (RaDuce) par chemin.
Réduction du MTTR	Démontré sur banc	À quantifier en production.
Réduction des coûts / appels Tier 1	Attendu	Bénéfice projeté, non encore mesuré.
Confidentialité (IA locale)	Réalisé	Aucune donnée abonné envoyée à un tiers.

pannes traité, quoique représentatif des incidents les plus fréquents, ne couvre pas encore l'ensemble du spectre (par exemple les anomalies applicatives ou de qualité de service fine).

6.5 Analyse de sécurité

La sécurité a été considérée dès la conception. L'authentification des abonnés repose sur un code PIN dont seule l'empreinte bcrypt est stockée. L'exécution 100 % locale du LLM garantit qu'aucune donnée d'abonné ne quitte l'infrastructure de l'opérateur. Surtout, la couche d'action applique une **liste blanche** : seuls les rôles explicitement déclarés éditables peuvent être écrits, ce qui interdit toute modification d'un paramètre sensible ou en lecture seule, même si le raisonnement du LLM le suggérait. Cette conception « sûre par construction » limite la surface d'erreur d'un agent autonome.

6.6 Scalabilité et maintenabilité

L'architecture en microservices confère au système une bonne scalabilité horizontale : chaque service peut être répliqué indépendamment selon la charge. L'externalisation de la connaissance dans le graphe est un atout de maintenabilité majeur : l'ajout d'un nouveau modèle de routeur ou l'adaptation à une mise à jour de firmware se traduit par une modification de données (nœuds et relations), sans toucher au code du moteur. Le faible couplage entre services facilite par ailleurs les évolutions et les tests.

6.7 Perspectives

Les limites identifiées dessinent des perspectives claires. À court terme, le raccordement à un GenieACS réel piloté par des passerelles physiques constituerait la validation grandeur nature. À moyen terme, le *fine-tuning* d'un modèle de langage spécialisé « télécom » à faible empreinte améliorerait la précision tout en réduisant les coûts. À plus long terme, l'ajout d'une **maintenance prédictive** — détection des coupures par apprentissage sur séries temporelles — ferait passer le système d'une posture réactive à une posture proactive, et l'ouverture de **canaux multimodaux** (vocal, téléphonique) élargirait l'accessibilité du support. Une autre piste consisterait à doter le système d'une *mémoire des incidents* : en conservant l'historique des pannes et des correctifs appliqués, l'agent pourrait détecter des récurrences, suggérer des causes profondes et alimenter un tableau de bord de fiabilité par modèle d'équipement. Le graphe s'y prête naturellement, puisqu'il suffirait d'y ajouter des relations temporelles entre incidents et équipements, transformant progressivement la base de connaissances en un véritable observatoire de la qualité du parc.

Ces perspectives partagent une même logique d'évolution : enrichir le système par la *donnée* plutôt que par le *code*. La maintenance prédictive, la mémoire des incidents ou l'ouverture à de nouveaux constructeurs se traduisent toutes par l'ajout de nœuds, de relations ou de séries temporelles au graphe, sans refonte du moteur. Cette propriété n'est pas fortuite : elle découle directement du choix fondateur d'externaliser la connaissance. En ce sens, l'architecture proposée n'est pas seulement une solution à un problème donné, mais une *plateforme* sur laquelle des capacités nouvelles peuvent se greffer progressivement.

On peut enfin envisager, à plus long terme, une boucle d'*amélioration continue* dans laquelle les résolutions validées par les ingénieurs viendraient renforcer la base de connaissances, et les rares échecs signaler les zones à enrichir. Le système apprendrait alors non pas en réentraînant un modèle, mais en affinant la connaissance structurée qu'il exploite — une forme d'apprentissage transparente, traçable et maîtrisée, particulièrement adaptée à un contexte industriel exigeant en fiabilité.

6.8 Conclusion

Ce chapitre a dressé un bilan honnête du projet : des objectifs techniques largement atteints, des bénéfices métiers démontrés sur banc et restant à quantifier en production, des limites clairement identifiées et des perspectives d'évolution concrètes.

6.9 Retour d'expérience

Au-delà des résultats techniques, ce projet a constitué une expérience formatrice sur plusieurs plans. Sur le plan **technique**, il a permis de manipuler un large spectre de technologies — LLM locaux, bases de données orientées graphe, protocoles d'orchestration d'outils, télégestion réseau, conteneurisation — et surtout d'apprendre à les faire coopérer au sein d'une architecture cohérente. Sur le plan **méthodologique**, l'application du 2TUP a montré l'intérêt de séparer les branches fonctionnelle et technique et de livrer par itérations, ce qui a considérablement réduit les risques d'intégration. Sur le plan **professionnel**, l'immersion au sein de Tritux a offert un aperçu concret des exigences de l'ingénierie logicielle en entreprise : rigueur, documentation, souci de la maintenabilité et attention portée à la valeur métier. La principale leçon retenue est peut-être qu'une innovation n'a de valeur que si elle est *fiable* et *maintenable* : c'est cette conviction qui a guidé les choix de conception les plus structurants, en particulier l'externalisation de la connaissance dans le graphe et la vérification systématique des actions.

6.10 Synthèse des contributions

Il est utile, en conclusion de cette évaluation, de récapituler les apports concrets du travail. La première contribution est la **résolution rôle** → **chemin** adossée au graphe, qui rend la connaissance véritablement déterminante et non plus décorative : c'est elle qui permet à un même correctif logique de s'appliquer correctement quel que soit le constructeur. La deuxième est le **statut dérivé**, qui interdit les fausses résolutions en recalculant l'état à partir des faits après chaque action. La troisième est la **couche d'action bornée** par liste blanche, qui encadre la liberté du LLM dans des limites sûres. La quatrième est l'**observabilité** du raisonnement, qui transforme l'IA en assistant auditable. Prises ensemble, ces contributions dessinent un patron d'ingénierie — connaissance structurée, action bornée, vérification systématique — transposable au-delà du seul support ISP.

6.11 Positionnement par rapport aux approches existantes

Il est éclairant de situer la solution par rapport aux alternatives couramment rencontrées. Un **système à base de scripts** (arbres de décision, FAQ dynamiques) est prévisible mais rigide : il ne comprend que ce qui a été prévu et n'agit pas sur l'équipement. Un **chatbot RAG documentaire** comprend le langage naturel et répond avec des sources,

mais s'arrête à la recommandation : il laisse l'abonné ou l'agent exécuter la correction. Une **automatisation par règles côté ACS** peut agir, mais elle est aveugle au langage de l'abonné et coûteuse à maintenir face à la diversité des constructeurs. Notre solution se distingue en réunissant les qualités de ces approches tout en évitant leurs limites : elle comprend (comme le chatbot), elle agit (comme l'automatisation ACS) et elle reste maintenable (grâce à l'externalisation de la connaissance dans le graphe).

Ce positionnement explique pourquoi une simple juxtaposition des outils existants n'aurait pas suffi. Brancher un chatbot sur un ACS sans couche de résolution reviendrait à donner la parole à un système sans lui donner la connaissance exacte nécessaire pour agir sans risque. La valeur du projet réside précisément dans la *colonne vertébrale* qui relie ces mondes : le graphe, qui porte à la fois le sens métier et la réalité technique, et la boucle de vérification, qui garantit l'honnêteté du résultat.

6.12 Enseignements généralisables

Plusieurs enseignements de ce projet dépassent son cadre immédiat. D'abord, dans un système agentique, la fiabilité ne vient pas du modèle seul mais de l'*environnement* qu'on lui construit : qualité de la connaissance récupérée, restriction des actions, vérification des effets. Ensuite, externaliser la connaissance hors du code est un puissant levier de maintenabilité : le système s'enrichit par les données plutôt que par le code. Enfin, la confiance dans un système autonome se gagne par la *transparence* : rendre le raisonnement observable transforme une boîte noire en un collaborateur que l'on peut superviser. Ces principes, illustrés ici sur le support ISP, constituent des repères pour concevoir d'autres agents opérationnels responsables.

Conclusion générale et perspectives

Ce projet de fin d'études a abouti à la conception et à la réalisation d'une **plateforme d'automatisation du support ISP** qui répond à la problématique posée : transformer la documentation technique passive d'un opérateur en une intelligence actionnable, capable de diagnostiquer et de corriger automatiquement les pannes des équipements d'abonnés.

Sur le plan des **réalisations**, plusieurs résultats concrets ont été atteints : la modélisation réussie de guides techniques complexes en un graphe de connaissances Neo4j fonctionnel ; la mise en œuvre d'une boucle agentique complète d'appel d'outils via le Model Context Protocol ; des pipelines de raisonnement LLM localisés et à faible latence ; et le déploiement d'une interface double, intuitive, à destination des abonnés comme des ingénieurs. La contribution centrale — la **résolution rôle** → **chemin TR-069** adossée au graphe — rend la connaissance véritablement déterminante : le même correctif logique s'applique au chemin correct propre à chaque constructeur, et le statut dérivé garantit qu'aucune action ne peut « réussir fausement ».

Sur le plan des **bénéfices attendus**, la solution vise, côté opérateur, une réduction significative du volume d'appels de premier niveau, des économies sur les opérations techniques et une diminution des interventions terrain ; côté abonné, un triage technique immédiat 24/7, des correctifs instantanés et un suivi transparent des incidents.

Ce travail comporte aussi des **limites** qu'il convient d'énoncer clairement. Le banc d'essai repose sur un serveur ACS *simulé* au-dessus de MongoDB (fidèle à l'arborescence TR-069) ; le raccordement à un serveur GenieACS réel piloté par des CPE physiques constituerait l'étape de validation suivante. De même, les gains quantitatifs (taux de résolution automatique, MTTR, taux d'escalade) restent à mesurer systématiquement sur un déploiement à plus grande échelle.

Ces limites dessinent naturellement les **perspectives** d'évolution : (i) le passage du banc d'essai simulé à des passerelles réseau réelles ; (ii) le *fine-tuning* d'un LLM télécom dédié à faible empreinte ; (iii) la maintenance prédictive par détection des coupures sur séries temporelles ; et (iv) l'ouverture de canaux de support multimodaux (vocal, téléphonique).

En définitive, ce projet démontre que la combinaison du GraphRAG, des cadres d'IA agentique et des protocoles standards de gestion à distance crée une couche intelligente et auto-réparatrice pour les réseaux d'accès — une architecture fiable, prête à absorber la montée en charge du support télécom de nouvelle génération.

Plus largement, ce travail illustre une conviction qui dépasse le seul cas du support ISP : à l'ère des grands modèles de langage, la valeur ne réside pas uniquement dans la puissance brute des modèles, mais dans la qualité de la connaissance qu'on leur donne à manipuler et dans la sûreté des actions qu'on les autorise à entreprendre. En structurant la connaissance dans un graphe et en encadrant l'action par une couche vérifiée, on transforme un modèle généraliste en un agent spécialisé, fiable et auditable. Ce schéma — connaissance structurée, action bornée, vérification systématique — constitue un patron réutilisable pour de nombreux domaines où l'IA doit non seulement conseiller, mais agir. À l'heure où l'intelligence artificielle générative suscite autant d'enthousiasme que d'inquiétude, ce type d'encadrement — qui autorise l'action tout en la bornant et en la vérifiant — apparaît comme une voie prometteuse pour déployer des agents autonomes de manière responsable. Le support télécom, avec ses protocoles standardisés et ses actions bien définies, en constitue un terrain d'application idéal, mais nombre d'autres domaines opérationnels pourraient bénéficier de la même approche.

Bibliographie

- [1] P. Lewis *et al.*, « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks », *NeurIPS*, 2020.
- [2] A. Vaswani *et al.*, « Attention Is All You Need », *NeurIPS*, 2017.
- [3] Broadband Forum, *TR-069 : CPE WAN Management Protocol (CWMP)*, Issue 1 Amendment 6, 2018.
- [4] Anthropic, *Model Context Protocol — Specification*, <https://modelcontextprotocol.io>, consulté en 2025.
- [5] *Universal Tool Calling Protocol (UTCP)*, documentation en ligne, consulté en 2025.
- [6] Neo4j Inc., *The Neo4j Cypher Manual*, <https://neo4j.com/docs/>, consulté en 2025.
- [7] Ollama — *Run large language models locally*, <https://ollama.com>, consulté en 2025.
- [8] Qwen Team, *Qwen2.5 Technical Report*, 2024.
- [9] *GenieACS — Open source TR-069 ACS*, <https://genieacs.com>, consulté en 2025.
- [10] Docker Inc., *Docker Documentation*, <https://docs.docker.com>, consulté en 2025.
- [11] Microsoft Research, « From Local to Global: A GraphRAG Approach to Query-Focused Summarization », 2024.

ANNEXE A

Orchestration des microservices (Docker Compose)

Cette annexe présente le fichier `docker-compose.yml` qui déclare l'ensemble des microservices de la plateforme, leurs images, leurs ports et le réseau `mypfe-net` qui les relie. Il illustre concrètement l'architecture physique décrite au chapitre 4.

```
1  services:
2
3    mongodb:
4      image: mongo:7
5      container_name: mypfe-mongodb
6      restart: unless-stopped
7      ports:
8        - "27018:27017"
9      volumes:
10       - mongodb_data:/data/db
11      networks:
12       - mypfe-net
13
14    ollama:
15      image: ollama/ollama:latest
16      container_name: mypfe-ollama
17      restart: unless-stopped
18      ports:
19        - "11434:11434"
20      volumes:
21       - ollama_data:/root/.ollama
22      networks:
23       - mypfe-net
24
25    mongo-seed:
26      build:
27        context: .
28        dockerfile: Dockerfile.seed
29      container_name: mypfe-mongo-seed
30      depends_on:
31       - mongodb
32      environment:
33        MONGO_URI: mongodb://mongodb:27017
34        MONGO_DB: mypfe
35      networks:
36       - mypfe-net
37      restart: "no"
38
39    neo4j-agent:
40      build:
41        context: .
```

```
42     dockerfile: Dockerfile.neo4j-agent
43     container_name: mypfe-neo4j-agent
44     restart: unless-stopped
45     ports:
46     - "8000:8000"    # also used by code-mode-mcp on Windows via localhost:8000
47     environment:
48     NEO4J_URI: bolt://host.docker.internal:7687    # your Windows Neo4j Desktop
49     NEO4J_USER: neo4j
50     NEO4J_PASSWORD: mahdi123
51     NEO4J_DATABASE: neo4j
52     SERVER_PORT: "8000"
53     BASE_URL: http://neo4j-agent:8000
54     healthcheck:
55     test: ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"]
56     interval: 10s
57     timeout: 5s
58     retries: 5
59     start_period: 10s
60     networks:
61     - mypfe-net
62
63     code-mode:
64     build:
65     context: .
66     dockerfile: Dockerfile.code-mode
67     container_name: mypfe-code-mode
68     restart: unless-stopped
69     ports:
70     - "8010:8010"
71     depends_on:
72     neo4j-agent:
73     condition: service_healthy
74     environment:
75     NEO4J_AGENT_URL: http://neo4j-agent:8000
76     CODE_MODE_PORT: "8010"
77     networks:
78     - mypfe-net
79
80     genie:
81     build:
82     context: .
83     dockerfile: Dockerfile.genie
84     container_name: mypfe-genie
85     restart: unless-stopped
86     ports:
87     - "8001:8001"
88     depends_on:
89     - mongodb
90     environment:
91     MONGO_URI: mongodb://mongodb:27017
92     MONGO_DB: mypfe
93     GENIE_PORT: "8001"
94     GENIE_BASE_URL: http://genie:8001
95     CUSTOMER_API: http://customer:8003
96     networks:
97     - mypfe-net
98
99     raduce:
100    build:
```

```
101     context: .
102     dockerfile: Dockerfile.raduce
103     container_name: mypfe-raduce
104     restart: unless-stopped
105     ports:
106     - "8002:8002"
107     depends_on:
108     - mongodb
109     environment:
110     MONGO_URI: mongodb://mongodb:27017
111     MONGO_DB: mypfe
112     RADUCE_PORT: "8002"
113     RADUCE_BASE_URL: http://raduce:8002
114     networks:
115     - mypfe-net
116
117     customer:
118     build:
119     context: .
120     dockerfile: Dockerfile.customer
121     container_name: mypfe-customer
122     restart: unless-stopped
123     ports:
124     - "8003:8003"
125     depends_on:
126     - mongodb
127     environment:
128     MONGO_URI: mongodb://mongodb:27017
129     MONGO_DB: mypfe
130     NEO4J_API: http://neo4j-agent:8000
131     CUSTOMER_PORT: "8003"
132     CUSTOMER_BASE_URL: http://customer:8003
133     networks:
134     - mypfe-net
135
136     mypfe-app:
137     build:
138     context: .
139     dockerfile: Dockerfile.mypfe-app
140     container_name: mypfe-app
141     restart: unless-stopped
142     ports:
143     - "5000:5000"
144     depends_on:
145     - neo4j-agent
146     - genie
147     - raduce
148     - customer
149     - ollama
150     environment:
151     OLLAMA_URL: http://ollama:11434
152     NEO4J_API: http://neo4j-agent:8000
153     GENIE_API: http://genie:8001
154     RADUCE_API: http://raduce:8002
155     CUSTOMER_API: http://customer:8003
156     GENIE_MCP: http://genie:8001/mcp/
157     RADUCE_MCP: http://raduce:8002/mcp/
158     CUSTOMER_MCP: http://customer:8003/mcp/
159     GMAIL_USER: ${GMAIL_USER:-}
```

```
160     GMAIL_APP_PASSWORD: ${GMAIL_APP_PASSWORD:-}
161     GMAIL_SMTP_HOST: smtp.gmail.com
162     GMAIL_SMTP_PORT: "587"
163     PYTHONUNBUFFERED: "1"
164     networks:
165       - mypfe-net
166
167     router-agent:
168       build:
169         context: .
170         dockerfile: Dockerfile.router-agent
171       container_name: mypfe-router-agent
172       restart: unless-stopped
173       ports:
174         - "5004:5004"
175       depends_on:
176         - neo4j-agent
177         - ollama
178         - code-mode
179       environment:
180         OLLAMA_URL: http://ollama:11434
181         FASTAPI_URL: http://neo4j-agent:8000
182         CODE_MODE_URL: http://code-mode:8010
183       networks:
184         - mypfe-net
185
186     volumes:
187       mongodb_data:
188       ollama_data:
189
190     networks:
191       mypfe-net:
192         driver: bridge
```

Listing A.1. docker-compose.yml — déclaration des microservices.

ANNEXE B

Extrait du moteur de résolution et de diagnostic

Pour illustrer la mise en œuvre de la contribution centrale du projet, cet extrait — volontairement réduit — montre le résolveur rôle → chemin (qui interroge le graphe puis se replie sur une carte locale) et le cœur de la fonction de diagnostic déterministe, laquelle trace au passage les chemins TR-069 réellement lus. Le code complet est disponible dans le dépôt du projet.

```
1  # --- Résolveur rôle -> chemin TR-069 (extrait) ---
2  class Resolver:
3      """Resolve rôle -> path, live from the graph if reachable, else cached."""
4
5      def __init__(self, neo4j_api: str | None = None, use_graph: bool = True):
6          self.neo4j_api = (neo4j_api or NEO4J_API).rstrip("/")
7          self.use_graph = use_graph
8          self._cache: dict[tuple[str, str], str | None] = {}
9
10     @staticmethod
11     def normalize_model(model: str) -> str | None:
12         """Map a model string to a key in PATHS (exact, then fuzzy)."""
13         if not model:
14             return None
15         if model in PATHS:
16             return model
17         needle = model.lower().replace(" ", "").replace("-", "").replace("_", "")
18         for key in PATHS:
19             k = key.lower().replace(" ", "").replace("-", "").replace("_", "")
20             if needle in k or k in needle:
21                 return key
22         for key in PATHS:
23             # last resort: token overlap
24             ktoks = set(key.lower().replace("-", " ").replace("_", " ").split())
25             mtoks = set(model.lower().replace("-", " ").replace("_", " ").split())
26             if ktoks & mtoks:
27                 return key
28         return None
29
30     def _from_graph(self, model: str, rôle: str) -> str | None:
31         if not self.use_graph:
32             return None
33         try:
34             r = requests.get(f"{self.neo4j_api}/tools/resolve_path",
35                             params={"model": model, "rôle": rôle}, timeout=5)
36             if r.status_code == 200:
37                 return (r.json() or {}).get("path") or None
38         except Exception:
39             pass
```

```

39         return None
40
41     def _from_cache_map(self, model: str, role: str) -> str | None:
42
43     # --- Diagnostic pilote par le graphe (extrait) ---
44     def diagnose(model: str, params: dict, resolver: Resolver) -> dict:
45         """
46         Deterministic, graph-driven diagnosis.
47
48         Returns dict:
49             fault          : str
50             fix_tool       : str | None
51             fix_extra      : dict          (args for the fix)
52             paths_read     : list[role, path, value]  <- the GraphRAG trace
53         """
54         trace: list[dict] = []
55
56         def val(role):
57             v, path = _read(params, resolver, model, role)
58             if path is not None:
59                 trace.append({"role": role, "path": path, "value": v})
60             return v
61
62         rx      = val("rx_optical_power")
63         crc     = val("dsl_crc_errors")
64         conn    = str(val("ppp_connection_status") or "")
65         vlan    = val("vlan_id")
66         dns     = str(val("dns_servers") or "")
67         lasterr = str(val("last_connection_error") or "")
68         val("uptime") # informational, recorded in trace
69
70         def out(fault, fix, extra=None):
71             return {"fault": fault, "fix_tool": fix,
72                     "fix_extra": extra or {}, "paths_read": trace}
73
74         # 1) Optical signal too weak -- physical, escalate.
75         try:
76             if rx is not None and float(rx) < RX_ESCALATE_DBM:
77                 return out("weak_signal", "escalate_to_technician")
78         except (TypeError, ValueError):
79             pass
80
81         # 2) DSL line errors too high -- hardware, escalate.
82         try:
83             if crc is not None and int(crc) > CRC_ESCALATE:
84                 return out("hardware_fault", "escalate_to_technician")
85         except (TypeError, ValueError):
86             pass
87
88         # 3) VLAN misconfigured -- remote fix.
89         if "unconfigured" in conn.lower():
90             return out("wrong_vlan", "set_vlan", {"vlan_id": CORRECT_VLAN})
91         try:
92             if vlan is not None and int(vlan) not in (0, CORRECT_VLAN):
93                 return out("wrong_vlan", "set_vlan", {"vlan_id": CORRECT_VLAN})
94         except (TypeError, ValueError):
95             pass
96
97         # 4) DNS broken -- remote fix.

```

```
98 if dns == "0.0.0.0":
```

Listing B.1. Extrait de `mcp_engine.py` — resolution role vers chemin et diagnostic.

ANNEXE C

Extrait du moteur de remédiation (RaDuce)

Cet extrait illustre l'application d'un correctif par le service RaDuce : résolution du rôle en chemin, contrôle d'éditabilité, écriture puis recalcul du statut.

```
1  # --- RaDuce : application d'un correctif (extrait) ---
2  def apply_paths(serial: str, model: str, path_values: dict) -> dict:
3      """
4      Write {path: value} into router_states.parameters, then re-derive status
5      and fault from the resulting parameters. Returns the new (status, fault).
6      """
7      state = get_state(serial)
8      params = flatten_parameters(state.get("parameters", {}))
9      params.update(path_values)
10
11     sets = {
12         "parameters": params,
13         "status": E.derive_status(model, params, resolver),
14         "fault": E.diagnose(model, params, resolver)["fault"],
15         "last_updated": datetime.utcnow(),
16     }
17     db.router_states.update_one({"serial": serial}, {"$set": sets})
18     return {"status": sets["status"], "fault": sets["fault"]}
19
20
21 def write_role(serial: str, model: str, role: str, value, consequence: dict) -> dict:
22     """
23     Resolve role -> path, enforce the editable allowlist, write value + the
24     physical consequence params, re-derive status. The single code path every
25     config fix goes through.
26     """
27     if not E.is_writable_role(role):
28         return {"success": False,
29                 "error": f"Role '{role}' is read-only -- RaDuce cannot write it."}
30
31     path = resolver.resolve(model, role)
32     if not path:
33         return {"success": False,
34                 "error": f"No TR-069 path for role '{role}' on model '{model}'."}
35
36     # Resolve the consequence roles to paths too (so we stay path-keyed).
37     written = {path: value}
38     for c_role, c_val in consequence.items():
39         c_path = resolver.resolve(model, c_role)
40         if c_path:
41             written[c_path] = c_val
42
```

```
43     result = apply_paths(serial, model, written)
44     return {"success": result["status"] == "UP",
45            "tr069_path_written": path,
46            "tr069_value": value,
47            "new_status": result["status"],
48            "new_fault": result["fault"]}
```

Listing C.1. Extrait de `raduce_server.py` — application d'un correctif.

ANNEXE D

Extrait de la résolution GraphRAG (neo4j-agent)

L'extrait suivant montre la requête Cypher de résolution rôle → chemin exposée par l'agent Neo4j.

```
1  # --- neo4j-agent : resolution role -> chemin (extrait) ---
2      except ValueError as e:
3          return {"error": str(e)}
4      except Exception as e:
5          return {"error": f"Server error: {e}"}
6
7
8  @app.get("/tools/resolve_path")
9  def api_resolve_path(model: str = Query(..., description="Router model / id"),
10                      role: str = Query(..., description="Canonical role, e.g. vlan_id")):
11      """Resolve a canonical role to the real TR-069 path for a given router model.
12      Backed by the 'role' stamped on HAS_PARAM by migrate_roles.py. This is what
13      makes the knowledge graph load-bearing: RaDuce calls it to know which
14      vendor-specific path to write."""
15      print(f"[server] resolve_path: model={model!r} role={role!r}")
16      try:
17          rows = db("""
18              MATCH (r:Router)-[rel:HAS_PARAM {role: $role}]->(p:Parameter)
19              WHERE toLower(r.id) CONTAINS toLower($model)
20              OR toLower(r.product_class) CONTAINS toLower($model)
21              OR toLower(r.sheet_name) CONTAINS toLower($model)
22              RETURN p.path AS path, p.editable AS editable, p.label AS label
23              LIMIT 1
24              """, {"model": model, "role": role})
25          if not rows:
26              return {"model": model, "role": role, "path": None,
27                      "error": "No path stamped for this (model, role)."}
28          row = rows[0]
29          return {"model": model, "role": role, "path": row["path"],
30                  "editable": row.get("editable"), "label": row.get("label")}
31      except Exception as e:
32          return {"model": model, "role": role, "path": None, "error": f"Server error: {e}"}
33
34
35  # -----
36  # Tool implementations
37  # -----
38
39  def tool_list_routers() -> dict:
40      rows = cached_db("list_routers", """
41          MATCH (r:Router)
```

Listing D.1. Extrait de neo4j-agent-server.py — résolution de chemin.

ANNEXE E

Extrait d'initialisation des données (seed)

Cet extrait présente la structure de l'état initial d'un routeur, indexé par chemins TR-069, tel qu'inséré dans MongoDB.

```
1  # --- seed.py : etat initial d'un routeur (extrait) ---
2  """
3  seed.py  --  Seed MongoDB for MyPFE (graph-aligned, path-keyed)
4
5  What changed vs v1
6  -----
7  * Customers now use the REAL router models that exist in the Neo4j graph
8    (Huawei_8145, TP-LINK_VC220-G3v, NOKIA_1425, ...) so every model resolves.
9  * router_states.parameters is now keyed by REAL TR-069 paths (a real ACS
10    device tree), built from the role-based fault model in mcp_engine.
11  * Faults are assigned per technology (optical weak_signal only to GPON, DSL
12    CRC hardware faults only to DSL), so the seeded data is physically coherent.
13  * status is derived from the parameters, not hard-coded.
14
15  Run:
16      python seed.py
17  """
18
19  import os
20  from datetime import datetime, timedelta
21
22  import bcrypt
23  from dotenv import load_dotenv
24  from pymongo import MongoClient
25
26  import mcp_engine as E
27
28  load_dotenv()
29
30  MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
31  MONGO_DB  = os.getenv("MONGO_DB", "mypfe")
32  NEO4J_API = os.getenv("NEO4J_API", "http://neo4j-agent:8000")
33
34  client  = MongoClient(MONGO_URI)
35  db      = client[MONGO_DB]
36  resolver = E.Resolver(NEO4J_API)
37
38
39  def hash_pw(plain: str) -> str:
40      return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()
41
42
```

```
43 # phone, pin, name, fault, vendor, model(=graph id), serial, technology, plan
44 CUSTOMERS = [
45     ("0661234567", "1234", "Mohamed Ben Ali", "ppp_auth_failure", "Huawei", "Huawei_8145",
46     ↪ "SN-HW-1001", "GPON", "Fiber 100M"),
47     ("0662345678", "2345", "Fatima Zahra Mansour", "weak_signal", "Huawei", "Huawei_HG8245H5",
48     ↪ "SN-HW-1002", "GPON", "Fiber 200M"),
49     ("0663456789", "3456", "Karim Trabelsi", "healthy", "Huawei", "HG8145X7-10",
50     ↪ "SN-HW-1003", "GPON", "Fiber 50M"),
```

Listing E.1. Extrait de seed.py — état initial d'un routeur.

ANNEXE F

Glossaire

Cette annexe rassemble les définitions des principaux termes techniques employés dans ce rapport.

ACS (Auto-Configuration Server)

Serveur central qui gère à distance les équipements des abonnés via le protocole TR-069 : il lit et écrit leurs paramètres de configuration.

Agent (IA)

Programme fondé sur un modèle de langage qui perçoit un état, décide d'une action et l'exécute via des outils, puis observe le résultat et itère.

bcrypt

Algorithme de hachage de mots de passe adaptatif, intégrant un sel aléatoire et un facteur de coût, résistant aux attaques par force brute.

CPE (Customer Premises Equipment)

Équipement installé chez l'abonné (modem, ONT, passerelle domestique).

CWMP / TR-069

Protocole normalisé de gestion à distance des CPE, défini par le Broadband Forum.

Cypher

Langage de requête déclaratif de la base de données orientée graphe Neo4j.

Embedding (plongement)

Représentation vectorielle d'un texte capturant son sens, utilisée pour mesurer la similarité sémantique.

GraphRAG

Variante du RAG où la récupération s'effectue par parcours des relations d'un graphe de connaissances, préservant les dépendances.

JSON-RPC

Protocole léger d'appel de procédure à distance fondé sur des messages JSON ; support des échanges MCP.

LLM (Large Language Model)

Grand modèle de langage de type Transformer, entraîné sur d'immenses corpus, capable de comprendre et de générer du texte.

MCP (Model Context Protocol)

Standard connectant un LLM à des outils et données externes de manière uniforme et sûre.

MTTR (Mean Time To Repair)

Temps moyen nécessaire pour résoudre un incident ; indicateur clé de la qualité de support.

Neo4j

Système de gestion de base de données orientée graphe, stockant nœuds et relations typées.

NOC (Network Operations Center)

Centre d'exploitation réseau d'où les ingénieurs supervisent le parc et traitent les incidents.

ONT (Optical Network Terminal)

Terminal optique installé chez l'abonné sur un réseau GPON.

RAG (Retrieval-Augmented Generation)

Approche consistant à récupérer des informations externes avant de générer une réponse, afin de l'ancrer dans des faits.

Rôle canonique

Nom logique et stable d'un paramètre (par ex. `vlan_id`), indépendant du chemin TR-069 réel, propre à chaque constructeur.

Statut dérivé

État (UP/DOWN) d'un routeur recalculé à partir de ses paramètres, et non stocké tel quel.

Token

Unité élémentaire de texte (mot ou fragment de mot) manipulée par un LLM.

UTCP (Universal Tool Calling Protocol)

Alternative au MCP où l'agent appelle directement les points d'accès HTTP décrits dans un « manuel » d'outils.

VLAN (Virtual LAN)

Réseau local virtuel ; sa configuration correcte est indispensable à la connectivité WAN de l'abonné.



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454